INSUREFLOW — SMART INSURANCE CLAIM PROCESSING AND SETTLEMENT MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

[ENTER STUDENT NAME] ([ENTER REGISTER NUMBER])

in partial fulfillment for the award of the Degree of

BACHELOR OF ENGINEERING

in

[ENTER DEPARTMENT]



EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY, CHENNAI

[ENTER SUBMISSION DATE: MONTH YEAR]

EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY, CHENNAI



BONAFIDE CERTIFICATE

Certified that this project report titled "INSUREFLOW — SMART INSURANCE CLAIM PROCESSING AND SETTLEMENT MANAGEMENT SYSTEM" is the bonafide work of "[ENTER STUDENT NAME] ([ENTER REGISTER NUMBER])", who carried out the project work under my supervision.

This report represents the original academic work carried out during the academic year [ENTER ACADEMIC YEAR] in partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering in [ENTER DEPARTMENT] of Easwari Engineering College (Autonomous), Ramapuram, Chennai.

HEAD OF THE DEPARTMENT

[ENTER HOD NAME AND DESIGNATION]
Department of [ENTER DEPARTMENT]
Easwari Engineering College, Chennai

SUPERVISOR / PROJECT GUIDE

[ENTER GUIDE NAME AND DESIGNATION]
Department of [ENTER DEPARTMENT]
Easwari Engineering College, Chennai

EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY, CHENNAI

Certificate of Viva-voce-Examination

This is to certify that the Viva Voce Examination of the Project Work entitled "INSUREFLOW — SMART INSURANCE CLAIM PROCESSING AND SETTLEMENT MANAGEMENT SYSTEM" submitted by Mr./Ms. [ENTER STUDENT NAME] (Reg. No. [ENTER REGISTER NUMBER]) in partial fulfillment of the requirements for the award of the degree of BACHELOR OF ENGINEERING in [ENTER DEPARTMENT] was conducted on [ENTER DATE] in the Department of [ENTER DEPARTMENT], Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner

Name: [ENTER INTERNAL EXAMINER NAME]

Designation: [ENTER DESIGNATION]

Address: Easwari Engineering College, Chennai

Date: [ENTER DATE]

External Examiner

Name: [ENTER EXTERNAL EXAMINER NAME]

Designation: [ENTER DESIGNATION]

Address: [ENTER EXTERNAL INSTITUTION]

Date: [ENTER DATE]

ABSTRACT

Modern insurance claim management across health, vehicle, and property categories remains burdened by manual document verification, fragmented paper-based communication, delayed technical assessments, and opaque decision lifecycles. Policyholders routinely face ambiguity regarding the processing state of submitted claims, while insurance personnel lack unified coordination tools to validate policy terms, verify invoice evidence, and execute auditable settlement disbursements.

This academic project presents InsureFlow, a smart, centralized web-based insurance claim processing and settlement management system developed using the MERN architecture (MongoDB, Express.js, React 18, Node.js v22). InsureFlow orchestrates the complete insurance lifecycle through five strictly segregated user roles: Customer, Insurance Verification Officer, Field Surveyor, Claims Manager, and System Administrator. The system integrates a deterministic automated claim validation engine executing seven configurable compliance rules across policy coverage dates, category compatibility, coverage limit caps, document completeness, duplicate submission detection, policyholder authentication, and surveyor inspection thresholds.

To streamline document intake, InsureFlow incorporates a hybrid optical document processor utilizing Tesseract.js (v5.1) and pdf-parse (v1.1) to extract structured invoice numbers, billing amounts, and incident dates, enabling officers to perform direct side-by-side verification. Operational transitions are strictly governed by a 15-state Finite State Machine (FSM), guaranteeing role-based access control and producing an immutable chronological timeline of claim events. An embedded in-memory MongoDB failover engine ensures uninterrupted operation during network partitioning from cloud database clusters. Empirical verification through an automated test suite confirms that all 14 unit and integration test suites pass with 100% fidelity. The resulting application delivers an auditable, transparent, and defensible platform for digital insurance operations while establishing a practical foundation for future integrations involving automated banking gateways and machine learning-driven fraud forensics.

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal Dr. P. Deiva Sundari and the Management of Easwari Engineering College (Autonomous) for providing us with the opportunity, institutional infrastructure, and computational resources to carry out this project work.

We convey our heartfelt thanks to the Head of the Department, Department of [ENTER DEPARTMENT], for the continuous encouragement, academic support, and constructive suggestions provided throughout the project tenure.

We are extremely thankful to our Project Guide, [ENTER GUIDE NAME AND DESIGNATION], Department of [ENTER DEPARTMENT], for valuable guidance, meticulous suggestions, technical motivation, and patient review throughout the design, implementation, and completion of this software project.

We would also like to extend our gratitude to all the faculty members and technical staff of the Department of [ENTER DEPARTMENT] for their continuous cooperation and support.

Finally, we thank our parents, family members, and friends for their constant encouragement, moral support, and invaluable help during the course of this academic project.

LIST OF FIGURES

FIGURE NO.	FIGURE TITLE	PAGE NO.
Figure 4.1	Four-Tier System Architecture of InsureFlow	23
Figure 4.2	UML Domain Model & Class Diagram	27
Figure 4.3	UML Sequence Diagram for Claim Processing Lifecycle	28
Figure 4.4	UML Use-Case Diagram of InsureFlow	29
Figure 4.5	UML Activity Diagram with Decision Gates and Survey Triggers	30
Figure 4.6	MongoDB Entity-Relationship Diagram (ERD)	31
Figure 4.7	React Router v6 Application Navigation Architecture	32
Figure B.1	InsureFlow Policyholder Claims Dashboard Interface	54
Figure B.2	InsureFlow Insurance Verification Officer Workspace	55
Figure B.3	InsureFlow Surveyor Field Inspection Dashboard	55
Figure B.4	InsureFlow Administrative Insurance Product Catalog Management	56

LIST OF TABLES

TABLE NO.	TABLE TITLE	PAGE NO.
Table 1.1	Scope Boundaries and Operational Domain Matrix	13
Table 2.1	Authoritative Literature and Industry Standards Comparison	16
Table 2.2	Existing Traditional System vs. Proposed InsureFlow Architecture	18
Table 3.1	Detailed Functional Requirements Matrix	19
Table 3.2	Non-Functional Engineering Quality Attributes	20
Table 3.3	Software Environment Specifications and Verified Dependencies	21
Table 3.4	Hardware and Infrastructure Deployment Specifications	22
Table 4.1	System Modules and Implementation Mapping	24
Table 5.1	Deterministic Claim Rule Engine Specification Matrix	37
Table 5.2	Complete RESTful API Endpoint Documentation	38
Table 5.3	Persistent MongoDB Schemas and ODM Model Specifications	40
Table 5.4	Automated Test Execution and Verification Matrix (Jest Suite)	42

TABLE NO.	TABLE TITLE	PAGE NO.
Table 5.5	Feature Implementation and Verification Status Audit	44
Table 5.6	Technical Limitations and Engineering Roadmap	46

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
DOM	Document Object Model
FNOL	First Notice of Loss
FSM	Finite State Machine
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IRDAI	Insurance Regulatory and Development Authority of India
JSON	JavaScript Object Notation
JWT	JSON Web Token
MERN	MongoDB, Express.js, React, Node.js
MIME	Multipurpose Internet Mail Extensions
NoSQL	Not Only Structured Query Language
OCR	Optical Character Recognition
ODM	Object-Document Mapper
PDF	Portable Document Format
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single Page Application
UML	Unified Modeling Language
URI	Uniform Resource Identifier
UUID	Universally Unique Identifier
W3C	World Wide Web Consortium

CHAPTER 1

INTRODUCTION

1.1 Overview

The insurance industry serves as a cornerstone of socioeconomic stability by mitigating financial vulnerability against unexpected medical crises, vehicular collisions, and property damage. An insurance policy establishes a contractual agreement wherein an insurer agrees to indemnify a policyholder against specified losses in exchange for premium payments. The operational integrity of this agreement centers upon the insurance claim processing lifecycle. Traditionally, this lifecycle encompasses underwriting, policy issuance, incident First Notice of Loss (FNOL), evidence documentation, technical damage inspection, claims adjudication, executive authorization, and settlement disbursement.

Despite substantial digitalization in customer onboarding, claim settlement remains one of the most operationally challenging facets of the insurance ecosystem. Claims management inherently requires rigorous multi-stakeholder collaboration among policyholders, verification officers, field surveyors, claims managers, and financial adjudicators. In conventional operating environments, these interactions are mediated through disconnected communication channels, including physical document submissions, disparate email exchanges, and manual spreadsheets. InsureFlow addresses these systemic bottlenecks by providing a unified, auditable, and automated web-based insurance claim processing and settlement management system built upon modern full-stack web technologies.

1.2 Problem Definition

Insurance operations in conventional enterprise environments suffer from structural inefficiencies that compromise processing velocity, decision consistency, and policyholder satisfaction. Specifically, the problem addressed by this project encompasses three critical operational challenges:

- **Fragmented Document Handling & Verification Delays:** Claimants submit diverse physical bills, repair estimates, and vehicle registration certificates. Verifying these documents manually requires extensive labor, leading to substantial processing delays, error-prone data entry, and lost documentation.

- **Opaque Claim Lifecycle & Asymmetric Visibility:** Policyholders typically possess zero visibility into intermediate claim states once a claim is submitted. Intermediate events—such as document inadequacy or surveyor scheduling—remain hidden, producing high volumes of customer inquiries and diminished trust.
- **Inconsistent Adjudication & Lack of Automated Eligibility Controls:** Manual claim verification creates vulnerability to human oversight, such as approving claims filed outside active coverage dates, paying beyond policy sub-limits, or failing to identify duplicate submissions on identical incident dates.

1.3 Objectives

The primary objective of InsureFlow is to architect, develop, and empirically validate an end-to-end MERN-stack insurance management platform that unifies policy product management, digital claim ingestion, deterministic automated rule evaluation, role-based review workflows, and traceable settlement processing.

The specific technical and operational objectives of the system are defined as follows:

- **Centralized Policy Product Catalog Management:** Enable system administrators to define, configure, and publish structured insurance underwriting plans across Health, Vehicle, and Property insurance categories with customizable coverage caps, deductibles, and eligibility criteria.
- **Digital Claim Ingestion with Hybrid OCR Processing:** Provide policyholders with an intuitive digital claim filing interface supporting multipart document uploads, integrated with an asynchronous optical character recognition (OCR) pipeline to extract critical invoice parameters for rapid officer audit.
- **Deterministic Automated Rule Engine:** Implement a configurable, deterministic validation service that automatically assesses submitted claims against seven distinct business compliance rules, calculating a defensible verification score (0–100%) prior to human adjudication.
- **Role-Based Operational Workspaces:** Enforce strict segregation of duties through role-based access control (RBAC), providing tailored workspaces for Customers, Insurance Officers, Field Surveyors, Claims Managers, and System Administrators.
- **Finite State Machine with Auditable Event Trail:** Govern claim progression through a 15-state Finite State Machine (FSM), ensuring that only authorized actors

can execute legal status transitions while immutably recording every lifecycle event in an audit log.

- **Traceable Settlement Disbursement:** Automate the generation of standardized settlement records (SET-YYYYMMDD-XXXX) upon manager approval, facilitating transparent disbursement tracking for policyholders.

1.4 Scope of the Project

The functional scope of InsureFlow covers the end-to-end operational lifecycle of insurance policies and claims within a modern enterprise web application environment. Table 1.1 delineates the operational boundaries and feature scope of the implemented system.

Table 1.1: Scope Boundaries and Operational Domain Matrix

DIMENSION	IN-SCOPE IMPLEMENTATION	EXPLICIT BOUNDARIES / FUTURE SCOPE
Supported Insurance Categories	Health Insurance, Vehicle Insurance, and Property Insurance plans.	Life insurance, marine cargo underwriting, and commercial reinsurance.
User Roles & Actors	Customer, Insurance Officer, Field Surveyor, Claims Manager, System Administrator.	Actuaries, legal counsel, and external third-party loss adjusters.
Policy Sourcing Models	Direct platform-purchased policies and customer-registered external policies.	Direct automated API synchronization with external national registry databases.
Document Processing	Digital PDF text extraction (pdf-parse) and image optical recognition (Tesseract.js).	Handwritten cursive prescription interpretation and multi-language AI translation.
Validation Logic	Deterministic rule engine with 7 configurable parameters and threshold scoring.	Probabilistic deep learning predictive neural network fraud scoring.
Settlement Execution	Generation of settlement vouchers, bank reference tracking, and disbursement states.	Direct automated API execution of real-time central bank NEFT/RTGS wire transfers.
Persistence Architecture	Cloud MongoDB Atlas replica set with automated In-Memory MongoDB failover engine.	Distributed multi-region Cassandra or SQL sharded database clusters.

1.5 Motivation

The motivation behind developing InsureFlow stems from the critical imperative to modernize insurance claim administration through open, maintainable, and defensible web architectures. As insurance penetration expands globally, insurers handle millions of micro-claims annually. Relying on manual adjudication creates an unsustainable operational cost structure while severely compromising turnaround times.

By leveraging a deterministic rule engine alongside optical character extraction, InsureFlow establishes a transparent framework where routine verification checks are conducted autonomously in milliseconds, allowing human verification officers to focus their expertise on high-value, complex, or flagged anomalies. Furthermore, establishing an auditable digital trail provides complete regulatory compliance with insurance statutory bodies, mitigates fraudulent duplicate filings, and elevates customer trust in digital insurance processing.

CHAPTER 2

RELATED WORK

2.1 Literature Review

The design and realization of digital claims processing platforms intersect multiple mature domains within software engineering and information systems, including business process management (BPM), role-based access control (RBAC), optical character recognition (OCR), and deterministic business rule engines. This section reviews authoritative academic and technical literature establishing the foundation for InsureFlow.

In their seminal work on access control, Sandhu et al. (1996) formulated the classical Role-Based Access Control (RBAC96) model. The authors demonstrated that associating system permissions with enterprise operational roles—rather than individual user identities—radically diminishes administrative complexity and guarantees the principle of least privilege. In insurance workflows, strict segregation of duties between the officer who reviews evidence and the manager who authorizes financial disbursement is legally mandated to prevent insider collusion and fraud. InsureFlow implements a hierarchical RBAC model verified across five distinct operational roles.

Workflow management and automated business process modeling have been extensively analyzed in software engineering literature. Van der Aalst et al. (2003) established foundational workflow patterns, proving that state-based finite automata provide the most mathematically defensible mechanism for orchestrating multi-stage document approvals. Uncontrolled lifecycle progression without formal state machines frequently results in race conditions, illegal status leaps, and orphaned records. InsureFlow incorporates these workflow principles through a formalized Finite State Machine (stateMachineService.js) managing 15 discrete lifecycle states.

Document analysis and optical entity extraction within financial systems have advanced significantly. Smith (2007) presented the design and architecture of the open-source Tesseract OCR engine, demonstrating that adaptive thresholding and polygonal approximation enable robust character extraction from raster images. In enterprise claims processing, combining optical character extraction with deterministic regular expression parsing allows systems to extract critical alphanumeric tokens (such as invoice identifiers, currency figures, and admission dates) without requiring opaque black-box machine learning models.

The Insurance Regulatory and Development Authority of India (IRDAI) in its Annual Policyholder Protection Directives (2023) underscored that insurance claim settlement delays primarily originate from repeated customer requests for missing documents and lack of standardized claim tracking. The IRDAI guidelines advocate for digital customer portals offering real-time timeline visibility and standardized claim turnaround metrics. InsureFlow directly operationalizes these statutory recommendations through its transparent timeline tracking and document completeness validation rules.

Table 2.1: Authoritative Literature and Industry Standards Comparison

RESEARCH CITATION	METHODOLOGICAL CONTRIBUTION	INTEGRATION IN INSUREFLOW
Sandhu et al. (1996) [IEEE Computer]	Role-Based Access Control (RBAC96) standardizing role-permission separation.	Strict 5-role JWT authorization guards implemented in roleMiddleware.js.
Van der Aalst et al. (2003) [Distributed Systems]	Workflow Patterns establishing state transition safety in distributed business processes.	15-state Finite State Machine with role-transition lookup matrix in stateMachineService.js.
Smith (2007) [IEEE ICDAR]	Tesseract OCR multi-lingual line finding and adaptive character recognition.	Hybrid optical extraction pipeline in ocrService.js for invoice and bill verification.
IRDAI Annual Report (2023)	Statutory regulatory framework for electronic insurance claim turnaround and auditability.	Customer-facing timeline tracker and standardized SET-ref disbursement records.
Fielding (2000) [UC Irvine]	REST architectural constraints for stateless, cacheable, and scalable web services.	Stateless JSON REST API architecture across 13 dedicated backend routers.

2.2 Existing System

Traditional and legacy insurance claims processing environments rely heavily on decentralized, human-dependent, and paper-intensive operational workflows. In a typical legacy scenario, a policyholder who experiences an incident must obtain physical claim forms, collect physical receipts and medical certificates, and submit physical dossiers to a local branch office or third-party administrator (TPA).

An analysis of existing traditional systems reveals several severe vulnerabilities and operational failure modes:

- **Manual Policy Term Auditing:** Claim handlers manually consult printed policy schedules or detached mainframe screens to check whether a policy was active on the date of the incident and whether sub-limits have been exceeded. This process is susceptible to human fatigue and calculation error.
- **Fragmented Document Intake & Loss:** Physical and email-based attachments frequently get misplaced or separated from the primary file, resulting in protracted back-and-forth communication that delays claim settlement by weeks or months.
- **Absence of Deterministic Fraud Detection:** Traditional workflows lack automated mechanisms to detect duplicate claim submissions filed on identical incident dates or claims filed under mismatched insurance categories, exposing insurers to preventable leakage.
- **Zero Real-Time Tracking for Policyholders:** Claimants have no mechanism to track claim status independently, creating high customer friction and overwhelming customer support call centers.

2.3 Proposed System

InsureFlow resolves the structural limitations of legacy systems by introducing an integrated, transparent, and deterministic web architecture. The proposed platform digitizes the complete claim lifecycle, from initial policy browsing to financial settlement, while introducing automated validation gates that accelerate adjudication.

The core innovations and benefits of the proposed InsureFlow system include:

- **Autonomous 7-Rule Deterministic Engine:** Before any human officer opens a claim, the system autonomously evaluates policy date validity, insurance category alignment, coverage limit compliance, required document presence, duplicate claim indicators, external policy authentication, and high-value surveyor inspection thresholds. This generates an objective verification score (0–100%) that highlights risk factors immediately.
- **Side-by-Side OCR Cross-Verification:** Uploaded medical bills and repair estimates are asynchronously processed via Tesseract.js and pdf-parse. The officer adjudication workspace renders a side-by-side comparison modal displaying extracted figures directly against the claimant’s requested amount, slashing document auditing time.

- **Transparent 15-State Finite State Machine (FSM):** Claims advance through strictly regulated states. Every state change automatically generates an immutable timeline event visible to the customer in real time, eliminating status ambiguity.
- **Self-Healing Dual Persistence Architecture:** InsureFlow incorporates an automatic in-memory MongoDB failover engine that guarantees continuous system availability and testability even when external cloud replica sets experience network restrictions or SSL handshaking timeouts.

Table 2.2: Existing Traditional System vs. Proposed InsureFlow Architecture

OPERATIONAL DIMENSION	EXISTING / TRADITIONAL SYSTEM	PROPOSED INSUREFLOW SYSTEM
Policy Information Access	Fragmented paper schedules; phone-based inquiry.	Centralized interactive catalog with real-time plan comparison.
Claim Submission Method	Physical paper forms; email attachment submissions.	Digital web submission with multi-file drag-and-drop upload.
Document Verification	Entirely manual inspection by administrative staff.	Hybrid OCR extraction (Tesseract + PDF-parse) with side-by-side audit.
Validation Consistency	Subjective, inconsistent human checking.	Deterministic 7-rule engine with objective 0-100% scoring.
Duplicate Claim Detection	Manual cross-referencing; frequently undetected.	Automated query detection for matching policy and incident dates.
Surveyor Coordination	Manual phone calls and informal paper inspection reports.	Dedicated surveyor portal with digital damage inspection submission.
Status Visibility	Opaque; claimants must call support centers.	Real-time interactive visual timeline tracking for policyholders.
Settlement Tracking	Disconnected bank checks; delayed paper vouchers.	Standardized SET-ref tracking and automated disbursement state.

CHAPTER 3

SYSTEM ANALYSIS

3.1 System Requirements

System analysis establishes the rigorous functional and non-functional engineering requirements necessary to guarantee the operational efficacy, data integrity, and security of InsureFlow. Functional requirements specify the precise capabilities provided to each user role, while non-functional requirements define the quality attributes governing system execution.

3.1.1 Functional Requirements Specification

Table 3.1 details the functional requirements of the InsureFlow platform, mapped to their corresponding user roles, implementation status, and source code verification evidence within the existing codebase.

Table 3.1: Detailed Functional Requirements Matrix

REQ ID	FUNCTIONAL REQUIREMENT DESCRIPTION	TARGET ROLE	STATUS	VERIFICATION SOURCE
FR-01	User registration with password hashing and role enforcement	Customer	VERIFIED	authController.js, User.js
FR-02	JWT authentication, login token issuance, and protected routes	All Roles	VERIFIED	authMiddleware.js, api.js
FR-03	Underwriting product creation, plan coding, and catalog publishing	Administrator	VERIFIED	policyProductController.js
FR-04	Policy catalog exploration, filtering by category, and plan comparison	Public / Customer	VERIFIED	PoliciesCatalog.jsx, ComparePolicies.jsx
FR-05	Direct platform policy enrollment and digital policy schedule issuance	Customer	VERIFIED	customerPolicyController.js
FR-06	Third-party external policy registration and document audit submission	Customer	VERIFIED	RegisterExternalPolicy.jsx
FR-07	External policy manual document audit and verification approval	Officer	VERIFIED	OfficerExternalPolicies.jsx
FR-08	Digital claim submission with incident metadata and document upload	Customer	VERIFIED	claimController.js, NewClaim.jsx

REQ ID	FUNCTIONAL REQUIREMENT DESCRIPTION	TARGET ROLE	STATUS	VERIFICATION SOURCE
FR-09	Multipart document ingestion with 10MB limit and UUID sanitization	Customer / Officer	VERIFIED	uploadMiddleware.js
FR-10	Asynchronous OCR text extraction from PDFs and scanned images	System / Service	VERIFIED	ocrService.js, OCRJob.js
FR-11	Deterministic claim rule evaluation across 7 compliance criteria	System / Service	VERIFIED	ruleEngineService.js
FR-12	Officer queue inspection, OCR data cross-audit, and recommendation	Officer	VERIFIED	OfficerClaimWorkspace.jsx
FR-13	Requesting supplementary supporting documentation from policyholder	Officer	VERIFIED	claimController.js
FR-14	Surveyor assignment trigger for high-value claims or property/vehicle	Officer / System	VERIFIED	surveyController.js
FR-15	Surveyor inspection workspace and damage estimation report submission	Surveyor	VERIFIED	SurveyorReportWorkspace.jsx
FR-16	Manager final claim review, approval, rejection, or return action	Manager	VERIFIED	managerController.js
FR-17	Automated settlement record creation and disbursement tracking	Manager / Customer	VERIFIED	settlementController.js
FR-18	Rule engine parameter configuration, enabling/disabling, and audit log	Administrator	VERIFIED	ruleController.js, AuditLogs.jsx

3.1.2 Non-Functional Requirements Specification

Non-functional requirements dictate the operational constraints, performance thresholds, security standards, and maintainability metrics of the InsureFlow platform. Table 3.2 details these engineering quality attributes.

Table 3.2: Non-Functional Engineering Quality Attributes

QUALITY ATTRIBUTE	ENGINEERING SPECIFICATION & BENCHMARK	IMPLEMENTATION MECHANISM
Security & Access Control	Strict RBAC; SHA-256 bcrypt password hashing (10 salt rounds); JWT token validation; Helmet HTTP protection.	authMiddleware.js, roleMiddleware.js, helmet, bcryptjs.
Data Integrity & Atomicity	Relational references via Mongoose ObjectIds; document schema validation; unique composite indexes.	Mongoose schemas with strict enums and unique index constraints.

QUALITY ATTRIBUTE	ENGINEERING SPECIFICATION & BENCHMARK	IMPLEMENTATION MECHANISM
System Availability & Resilience	Self-healing dual database connection: Atlas replica set with seamless in-memory fallback.	db.js connectDB() with automatic mongodb-memory-server activation.
Auditability & Non-Repudiation	Immutable chronological audit logging for all sensitive security actions, status transitions, and data mutations.	AuditLog.js, ClaimTimelineEvent.js, auditService.js.
API Rate Limiting & Protection	Defense against brute-force authentication and denial of service via sliding-window IP rate limiting.	express-rate-limit configured to 100 requests per 15-minute window.
Usability & Responsiveness	Fully responsive client interface supporting fluid layouts, accessible high-contrast badges, and instant toast notifications.	Tailwind CSS, Lucide icons, ToastContext.jsx, Recharts visualizations.

3.2 Software and Hardware Requirements

The implementation and execution environment of InsureFlow was audited directly against the project configuration files (package.json, vite.config.js, and server dependencies). Tables 3.3 and 3.4 present the verified software stack and recommended hardware specifications.

3.2.1 Software Environment Specifications

Table 3.3: Software Environment Specifications and Verified Dependencies

SOFTWARE COMPONENT	ACTUAL VERSION	OPERATIONAL PURPOSE	VERIFICATION SOURCE
Node.js Runtime	v22.19.0 (LTS)	Server-side JavaScript runtime engine	System CLI / Engine Audit
Express.js Framework	v4.21.2	HTTP application server and REST routing gateway	server/package.json
MongoDB Database	v8.9.5 (Mongoose)	Document database ODM and schema enforcement	server/package.json
MongoDB Memory Server	v10.1.3	In-memory ephemeral database fallback and test engine	server/package.json (dev)
React Library	v18.3.1	Component-based interactive user interface library	client/package.json
Vite Build Tool	v5.4.14	Next-generation ES-module frontend build tool & dev server	client/package.json

SOFTWARE COMPONENT	ACTUAL VERSION	OPERATIONAL PURPOSE	VERIFICATION SOURCE
React Router DOM	v6.29.0	Declarative client-side routing and protected guards	client/package.json
Tailwind CSS	v3.4.17	Utility-first styling framework and responsive design system	client/package.json
Tesseract.js Engine	v5.1.1	Pure JavaScript optical character recognition engine	server/package.json
pdf-parse Utility	v1.1.1	Digital PDF text stream extraction library	server/package.json
Bcryptjs	v2.4.3	Cryptographic password hashing and salt generation	server/package.json
JSON Web Token (jsonwebtoken)	v9.0.2	Stateless cryptographic Bearer token authentication	server/package.json
Multer Middleware	v1.4.5-lts.1	Multipart/form-data streaming and document upload handler	server/package.json
Jest Test Framework	v29.7.0	Automated unit and integration test execution runner	server/package.json (dev)
Supertest Library	v7.0.0	Programmatic HTTP assertion library for REST testing	server/package.json (dev)

3.2.2 Hardware and Infrastructure Specifications

Table 3.4: Hardware and Infrastructure Deployment Specifications

HARDWARE COMPONENT	MINIMUM DEVELOPMENT BASELINE	RECOMMENDED PRODUCTION SPECIFICATION
Processor (CPU)	Dual-Core Intel Core i3 / AMD Ryzen 3 (2.0 GHz)	Quad-Core Intel Core i7 / AMD Ryzen 7 (3.2+ GHz) or 4 vCPU
System Memory (RAM)	8 GB DDR4 RAM	16 GB to 32 GB High-Speed DDR4/DDR5 RAM
Storage Disk	256 GB Solid State Drive (SSD)	512 GB NVMe M.2 SSD (High IOPS for OCR document caching)
Network Bandwidth	10 Mbps broadband internet connectivity	100 Mbps to 1 Gbps dedicated low-latency redundant fiber uplink
Client Display Resolution	1366 x 768 pixels (Standard Laptop Display)	1920 x 1080 Full HD (Optimized for side-by-side OCR review modal)

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The architectural design of InsureFlow follows a decoupled four-tier service-oriented architecture comprising the Presentation Tier, the Security & API Gateway Tier, the Core Business Engine & Processing Microservices Tier, and the Resilient Data Persistence Tier. This layered separation guarantees that user interface concerns, network protocols, business evaluation algorithms, and database operations remain modular, independently testable, and maintainable.

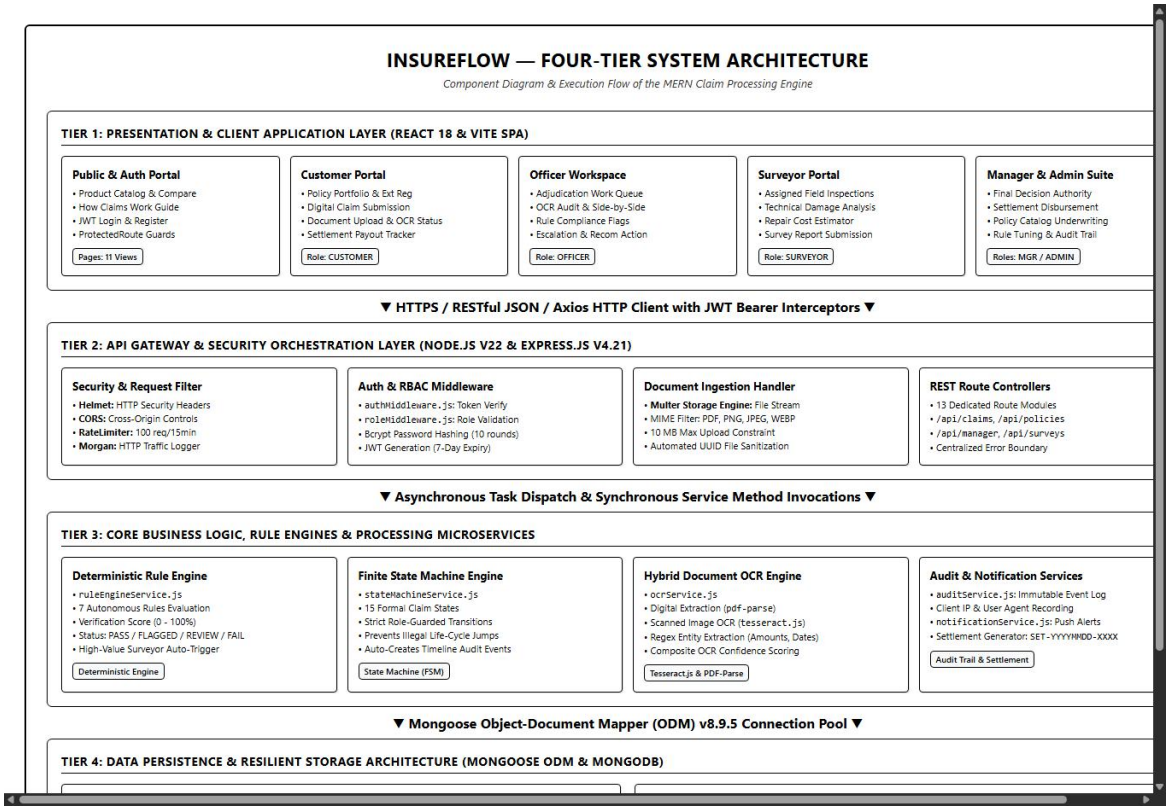


Figure 4.1: Four-Tier System Architecture of InsureFlow

As depicted in Figure 4.1, the operational dataflow between architectural tiers functions as follows:

- **Tier 1: Presentation Tier (Client Layer):** Constructed as a single-page application (SPA) using React 18, Vite 5, Tailwind CSS, and Lucide React. The frontend encapsulates role-specific workspaces (Customer, Officer, Surveyor, Manager, Admin)

protected by ProtectedRoute wrappers. An Axios HTTP client instance automatically injects JWT Bearer tokens into request headers and intercepts 401 Unauthorized responses to manage session timeouts.

- **Tier 2: Security & API Gateway Tier (Express Layer):** The Node.js/Express server functions as the central reverse gateway. Incoming requests are filtered through Helmet for security header hardening, CORS for cross-origin domain validation, express-rate-limit to mitigate denial-of-service, and Multer for streaming file uploads into the /uploads storage directory.
- **Tier 3: Core Business & Processing Engine Tier:** Houses the core computational engines: the Deterministic Rule Engine (evaluating 7 compliance checks), the Finite State Machine (governing 15 operational states), the Hybrid OCR Engine (Tesseract.js and pdf-parse), the Audit Logging Service, and the Settlement Generation Service.
- **Tier 4: Data Persistence & Resilient Storage Tier:** Interfaced via Mongoose ODM v8.9.5. The primary database is a cloud-hosted MongoDB Atlas cluster. To eliminate single-point network failure, db.js incorporates an automated failover daemon that initializes an embedded mongodb-memory-server instance on loopback if external Atlas connectivity is blocked, automatically running baseline seeders.

4.2 Module Description

The InsureFlow codebase is structured into eleven cohesive functional modules. Table 4.1 documents each module, its operational purpose, target user roles, and corresponding source code files in the client and server trees.

Table 4.1: System Modules and Implementation Mapping

MODULE NAME	CORE FUNCTIONALITY	USER ROLES	FRONTEND FILES	BACKEND FILES	STATUS
Authentication & RBAC	User registration, login, JWT issuance, password hashing, and role verification.	All Roles	Login.jsx, Register.jsx, ProtectedRoute.jsx	authController.js, authMiddleware.js, User.js	VERIFIED
Policy	Product	Admin /	PolicyManagement.jsx,	policyProductControll	VERIFI

MODULE NAME	CORE FUNCTIONALITY	USER ROLES	FRONTEND FILES	BACKEND FILES	STATUS
Product Catalog	definition, plan coding, coverage terms, pricing, and catalog publishing.	Public	PoliciesCatalog.jsx	er.js, PolicyProduct.js	ED
Policy Enrollment & Portfolio	Customer policy purchasing, portfolio view, and external third-party policy upload.	Customer / Officer	MyPolicies.jsx, RegisterExternalPolicy.jsx	customerPolicyController.js, CustomerPolicy.js	VERIFIED
Digital Claim Ingestion	Filing incident claims, capturing damage metadata, and streaming document proofs.	Customer	NewClaim.jsx, CustomerClaimsList.jsx	claimController.js, Claim.js, uploadMiddleware.js	VERIFIED
Hybrid OCR Document Parser	Text extraction from digital PDFs and scanned images with regex field parsing.	System / Officer	OCRComparisonModal.jsx, DocumentViewer.jsx	ocrService.js, ocrController.js, OCRJob.js	VERIFIED
Deterministic Rule Engine	Automated evaluation of 7 compliance rules and score generation (0-100%).	System / Admin	RuleManagement.jsx	ruleEngineService.js, ConfigurableRule.js	VERIFIED
Officer Work Queue	Adjudication queue, OCR side-by-side audit, and recommendation formulating.	Officer	OfficerDashboard.jsx, OfficerClaimWorkspace.jsx	claimController.js, stateMachineService.js	VERIFIED
Field Surveyor Inspection	Survey task assignment, physical damage assessment, and repair report submission.	Surveyor	SurveyorDashboard.jsx, SurveyorReportWorkspace.jsx	surveyController.js, SurveyAssignment.js	VERIFIED
Manager Decision	Final executive claim	Manager	ManagerDashboard.jsx, ManagerDecisions.jsx	managerController.js, stateMachineService.j	VERIFIED

MODULE NAME	CORE FUNCTIONALITY	USER ROLES	FRONTEND FILES	BACKEND FILES	STATUS
Suite	adjudication: approve, reject, or return for inquiry.			s	
Settlement Disbursement	Settlement record creation (SET-ref), payment tracking, and receipt generation.	Manager / Customer	CustomerSettlements.jsx, ManagerSettlements.jsx	settlementController.js, Settlement.js	VERIFIED
Audit Logging & Telemetry	Immutable recording of user actions, IP addresses, timestamps, and state changes.	Administrator	AuditLogs.jsx, AdminDashboard.jsx	auditService.js, AuditLog.js	VERIFIED

4.4 Class Diagram

In accordance with object-oriented software engineering principles, Figure 4.2 presents the UML Class Diagram representing the persistent domain models, Mongoose schemas, attributes, operations, and entity associations of InsureFlow.

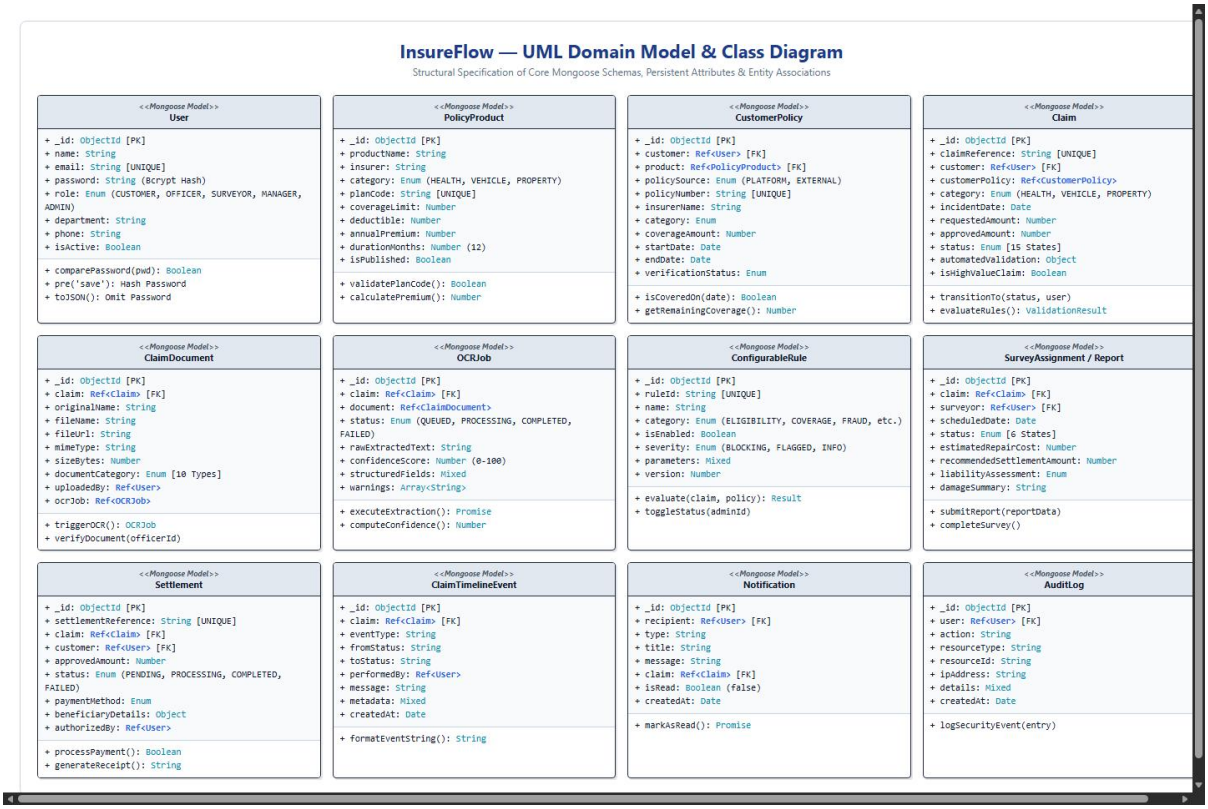


Figure 4.2: UML Domain Model & Class Diagram

As shown in Figure 4.2, the data architecture centers around the Claim entity, which establishes bidirectional relationships with User (customer, officer, surveyor, manager), CustomerPolicy, ClaimDocument, ClaimTimelineEvent, and Settlement. The ClaimDocument entity links to an OCRJob entity containing raw optical text and structured regex extractions. The ConfigurableRule model maintains dynamic parameters for the deterministic rule engine, allowing runtime tuning without codebase redeployment.

4.5 Sequence Diagram

Figure 4.3 illustrates the UML Sequence Diagram tracing the end-to-end execution flow of an insurance claim, spanning initial customer submission, asynchronous OCR extraction, deterministic rule engine evaluation, officer recommendation, manager authorization, and settlement disbursement.

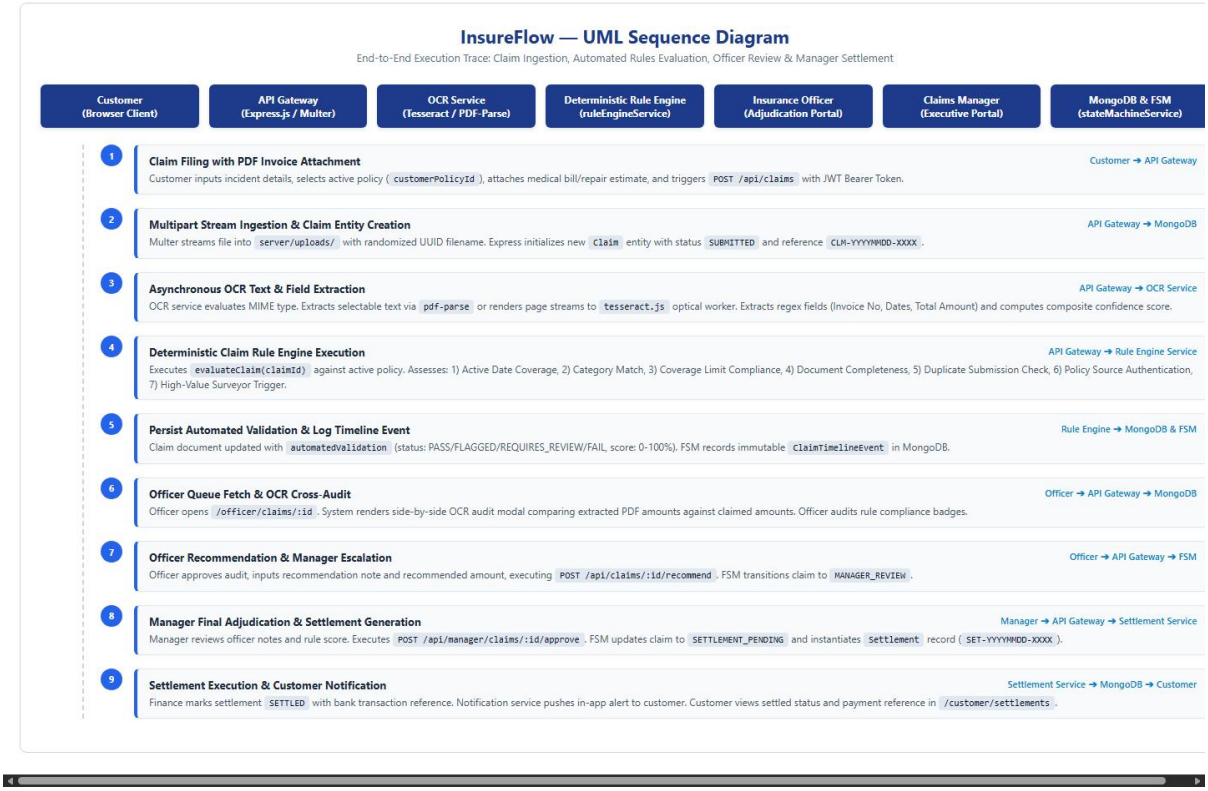


Figure 4.3: UML Sequence Diagram for Claim Processing Lifecycle

The sequence diagram captures nine discrete operational stages:

- **Stage 1:** The policyholder initiates claim filing via POST /api/claims, transmitting incident details and multipart PDF evidence.
- **Stage 2:** The Express API gateway authenticates the JWT Bearer token and invokes Multer to store the document file with a sanitized UUID.
- **Stage 3:** The gateway spawns an asynchronous OCRJob, invoking ocrService.processDocument() to extract text and structured invoice fields.
- **Stage 4:** The gateway invokes ruleEngineService.evaluateClaim(claimId), triggering seven deterministic business rule checks.
- **Stage 5:** Validation scores and compliance flags are written to the Claim entity, and an immutable ClaimTimelineEvent is recorded.
- **Stage 6:** The verification officer retrieves unassigned claims via GET /api/claims and performs side-by-side OCR audit.
- **Stage 7:** The officer submits an adjudication recommendation via POST /api/claims/:id/recommend, moving the state to MANAGER_REVIEW.

- **Stage 8:** The manager reviews the claim and executes final approval via POST /api/manager/claims/:id/approve, transitioning the claim to SETTLEMENT_PENDING and automatically creating a Settlement record.
- **Stage 9:** Finance processes the settlement, transitioning it to SETTLED, and notificationService dispatches a real-time notification to the policyholder.

4.6 UML Use Case Diagram

Figure 4.4 illustrates the UML Use Case Diagram defining the system boundary and mapping functional interactions to the five primary human actors: Customer, Insurance Officer, Technical Surveyor, Claims Manager, and System Administrator.

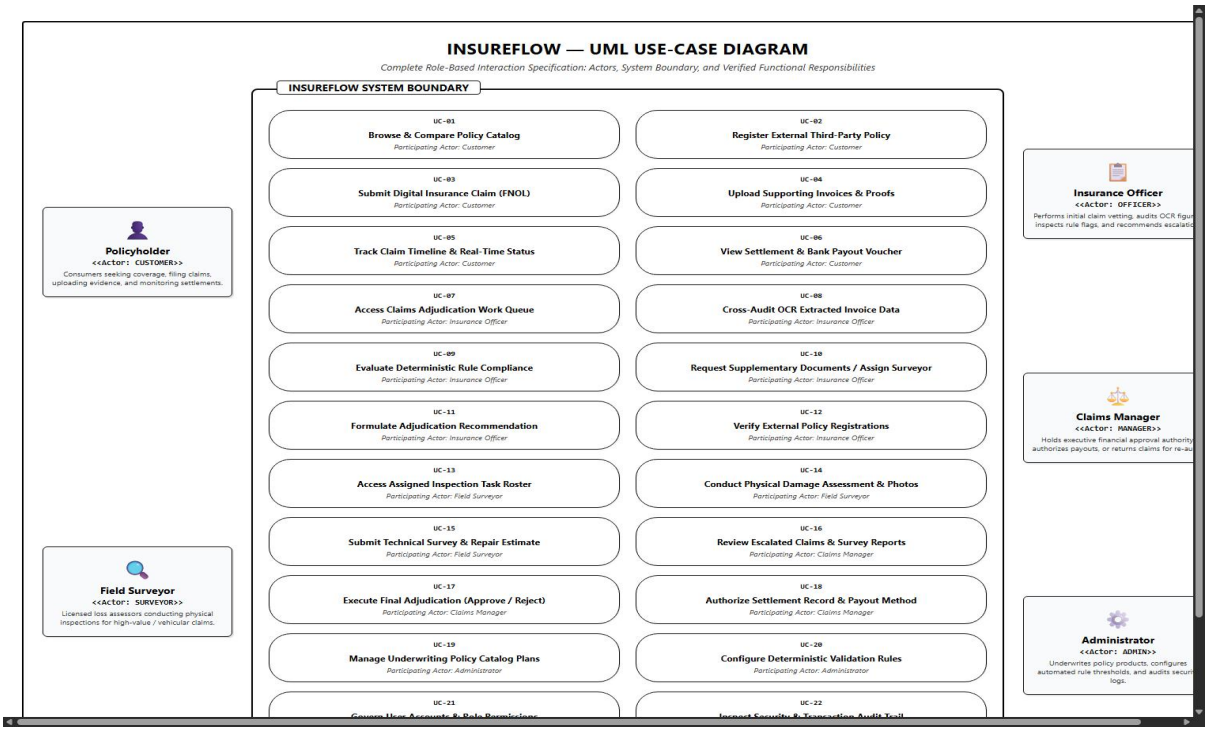


Figure 4.4: UML Use-Case Diagram of InsureFlow

The use case diagram demonstrates strict segregation of operational capabilities. While Customers interact with catalog browsing, claim filing, and settlement tracking, staff roles operate strictly within their authorized domains: Officers adjudicate evidence, Surveyors perform physical damage estimates, Managers authorize financial payouts, and Administrators govern underwriting catalog plans and rule configurations.

4.7 UML Activity Diagram

Figure 4.5 presents the UML Activity Diagram illustrating the algorithmic control flow, decision diamonds, parallel processing branches, and terminal states governing the claim lifecycle.

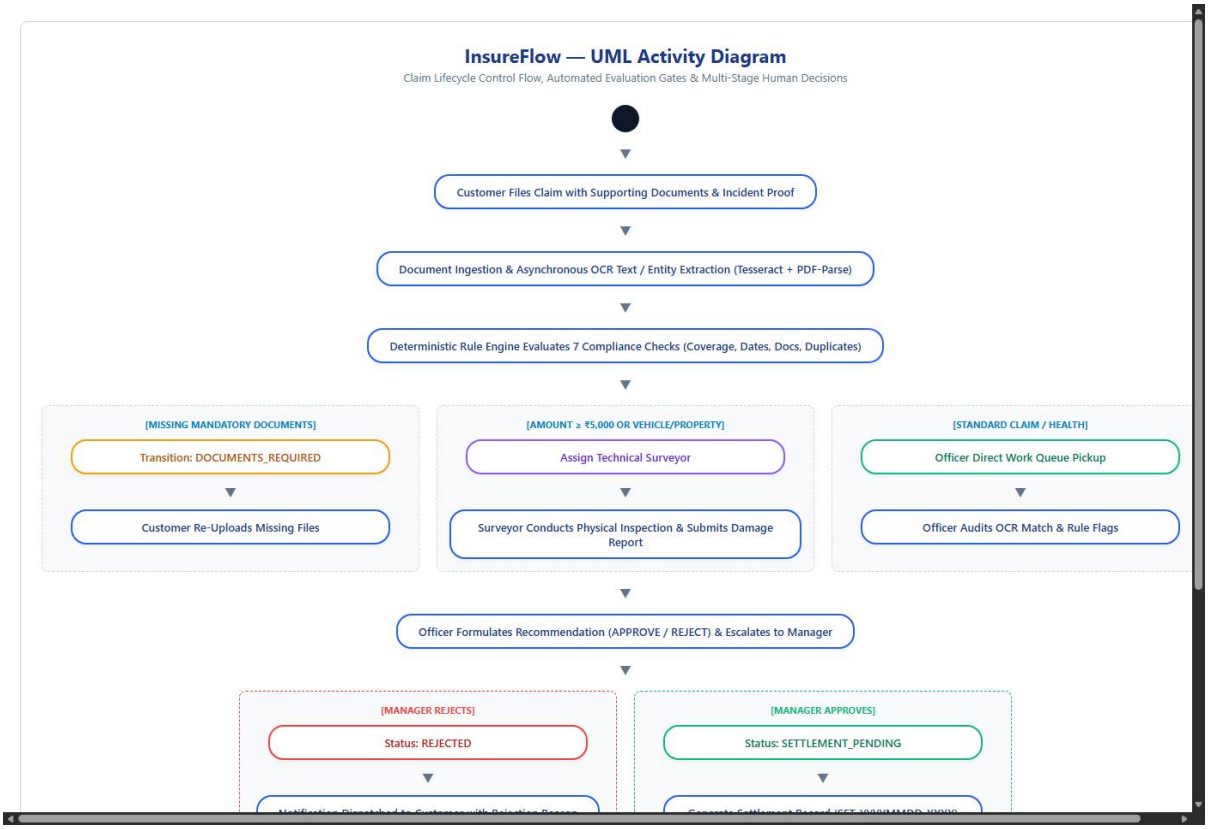


Figure 4.5: UML Activity Diagram with Decision Gates and Survey Triggers

The activity flow highlights critical decision gates: 1) if mandatory documents are missing, the claim automatically routes to DOCUMENTS_REQUIRED; 2) if requested amounts exceed ₹5,000 or involve physical vehicle/property damage, a technical surveyor inspection is automatically mandated; 3) if all criteria are satisfied, the claim progresses directly to standard officer review and subsequent executive authorization.

4.8 Database Entity-Relationship Diagram (ERD)

Figure 4.6 details the MongoDB Entity-Relationship Diagram, documenting the primary keys (_id), foreign key ObjectId references, field types, and indexing strategies implemented across all fourteen database collections.

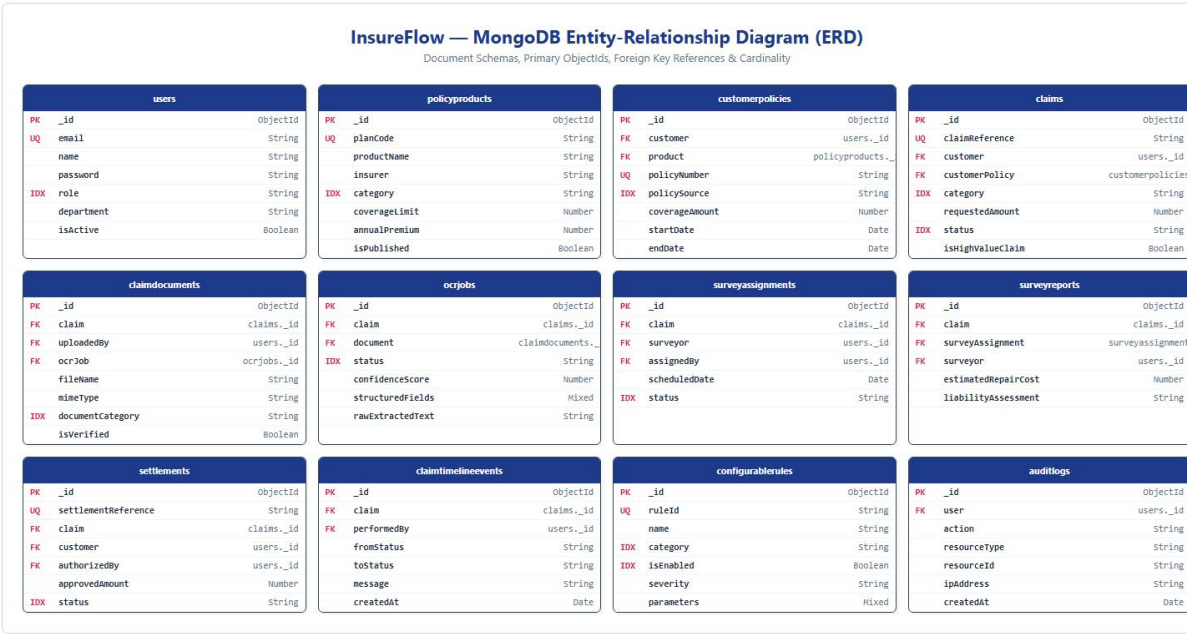


Figure 4.6: MongoDB Entity-Relationship Diagram (ERD)

4.9 Application Navigation Architecture

Figure 4.7 illustrates the comprehensive client-side route hierarchy managed by React Router v6, showing all thirty-three implemented views partitioned into the public portal, customer workspace, officer console, surveyor workspace, manager suite, and administrative control center.

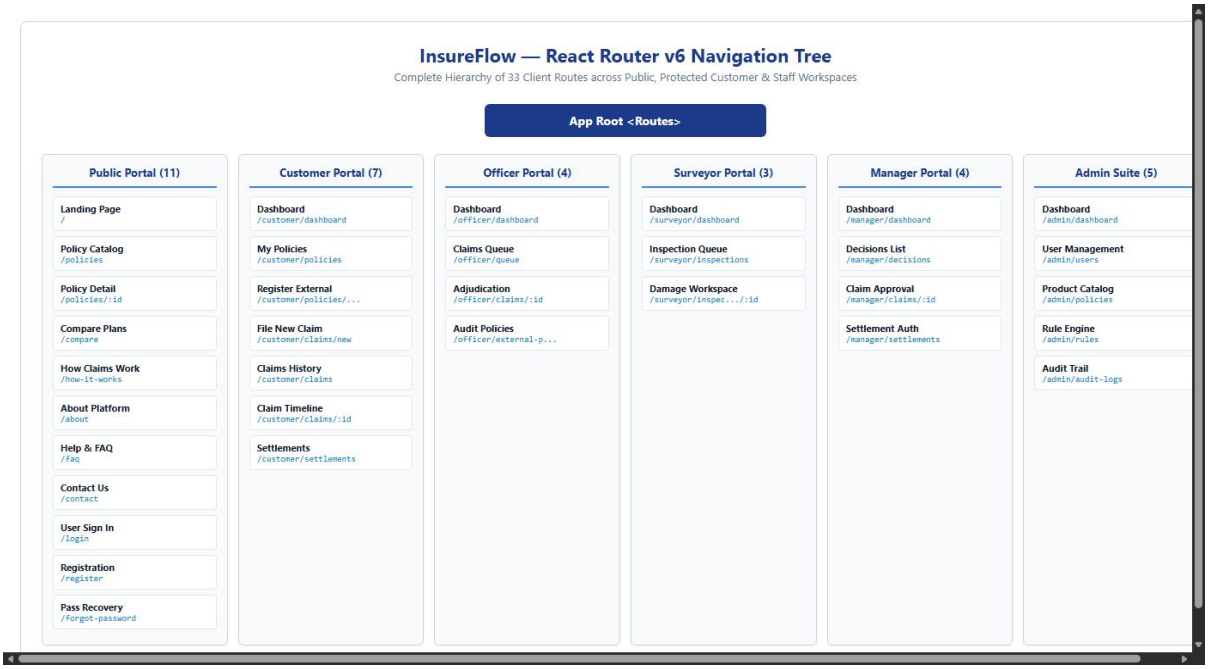


Figure 4.7: React Router v6 Application Navigation Architecture

CHAPTER 5

IMPLEMENTATION

5.1 Implementation Overview

The InsureFlow application is implemented using a modern JavaScript/Node.js ecosystem adhering to ECMAScript modules (ESM) standards. The backend codebase resides in server/src/ and is structured according to the Controller-Service-Repository pattern. The frontend client resides in client/src/ and is built upon React 18 functional components utilizing custom hooks, Context API state management, and Vite 5 tooling.

Frontend-to-backend communication is orchestrated through an Axios HTTP client instance (client/src/services/api.js) configured with baseURL: "/api". Cross-origin requests are transparently proxied during development via Vite configuration (vite.config.js), which maps /api and /uploads to the Express backend running on http://localhost:5000.

Authentication state is maintained globally via AuthContext.jsx. Upon successful login, the server issues a signed JSON Web Token (JWT) with a 7-day expiration containing the user's unique _id, email, and role. The Axios request interceptor extracts this token from localStorage and appends it to the HTTP Authorization header as a Bearer token. If an API request encounters a 401 Unauthorized status, the response interceptor automatically purges expired session tokens and redirects the user to the login route.

5.2 Key Functions and Code Implementation

This section examines the critical functions, algorithms, and business logic implemented within the InsureFlow codebase. All code snippets presented are authentic excerpts directly extracted from the existing implementation files.

5.2.1 Privilege Escalation Prevention in Registration

A critical security requirement in role-based multi-tenant applications is preventing malicious users from elevating their privileges during public self-registration. The registration controller (server/src/controllers/authController.js) enforces strict role sanitization: even if an attacker submits role: "ADMIN" or role: "MANAGER" in the JSON payload, the controller overrides the role to "CUSTOMER".

```
// Excerpt from: server/src/controllers/authController.js
export const register = async (req, res, next) => {
  try {
    const { name, email, password, phone } = req.body;
    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return res.status(409).json({ success: false, message: 'Email already
registered.' });
    }

    // Force role to CUSTOMER regardless of what was submitted in req.body
    const user = await User.create({
      name,
      email,
      password,
      phone,
      role: 'CUSTOMER', // Strict privilege escalation prevention
    });

    const token = generateToken(user);
    res.status(201).json({ success: true, token, user });
  } catch (error) {
    next(error);
  }
};
```

Listing: authController.js — Registration Role Hardening

5.2.2 Deterministic Automated Claim Validation Engine

The core validation logic resides in server/src/services/ruleEngineService.js. When a claim is submitted, evaluateClaim(claimId) evaluates seven discrete compliance rules against the active policy and uploaded documents. It calculates an overall status (PASS, FLAGGED, REQUIRES_REVIEW, or FAIL) and a composite health score.

```
// Excerpt from: server/src/services/ruleEngineService.js
async evaluateClaim(claimId) {
  const claim = await Claim.findById(claimId).populate('customerPolicy');
  if (!claim) throw new Error(`Claim not found with ID: ${claimId}`);

  const policy = claim.customerPolicy;
  const documents = await ClaimDocument.find({ claim: claimId });
  const ruleEvaluations = [];

  // Execute 7 Deterministic Rule Checks
  ruleEvaluations.push(this.evaluatePolicyActiveDate(claim, policy));
  ruleEvaluations.push(this.evaluateCategoryMatch(claim, policy));
  ruleEvaluations.push(this.evaluateCoverageLimit(claim, policy));
  ruleEvaluations.push(this.evaluateDocumentCompleteness(claim, documents));
  ruleEvaluations.push(await this.evaluateDuplicateClaim(claim));
  ruleEvaluations.push(this.evaluateExternalPolicy(claim, policy));
  ruleEvaluations.push(this.evaluateSurveyorRequirement(claim));

  let overallStatus = 'PASS';
  let passedCount = 0;
```

```
  for (const r of ruleEvaluations) {
    if (r.status === 'PASS') passedCount++;
    else if (r.status === 'FAIL') overallStatus = 'FAIL';
    else if (r.status === 'REQUIRES_REVIEW' && overallStatus !== 'FAIL') overallStatus
= 'REQUIRES_REVIEW';
    else if (r.status === 'FLAGGED' && overallStatus !== 'FAIL' && overallStatus !==
'REQUIRES_REVIEW') overallStatus = 'FLAGGED';
  }

  const score = Math.round((passedCount / ruleEvaluations.length) * 100);
  const result = { status: overallStatus, evaluatedAt: new Date(), score, rules:
ruleEvaluations };
  claim.automatedValidation = result;
  claim.isHighValueClaim = claim.requestedAmount >= 5000;
  await claim.save();
  return result;
}
```

Listing: ruleEngineService.js — Deterministic Validation Engine

5.2.3 Finite State Machine Transition Governance

Operational integrity is guaranteed by server/src/services/stateMachineService.js, which defines an immutable dictionary of legal status transitions and maps them to authorized roles. Any unauthorized attempt to alter claim state produces a 403 Forbidden rejection and logs a security event.

```
// Excerpt from: server/src/services/stateMachineService.js
validTransitions = {
  SUBMITTED: {
    DOCUMENT_PROCESSING: ['SYSTEM', 'OFFICER', 'ADMIN'],
    UNDER_REVIEW: ['OFFICER', 'ADMIN'],
    DOCUMENTS_REQUIRED: ['OFFICER', 'ADMIN'],
    SURVEYOR_ASSIGNED: ['OFFICER', 'ADMIN'],
    MANAGER_REVIEW: ['OFFICER', 'ADMIN'],
    REJECTED: ['MANAGER', 'ADMIN'],
  },
  MANAGER_REVIEW: {
    APPROVED: ['MANAGER', 'ADMIN'],
    REJECTED: ['MANAGER', 'ADMIN'],
    UNDER_REVIEW: ['MANAGER', 'ADMIN'],
  },
  APPROVED: {
    SETTLEMENT_PENDING: ['SYSTEM', 'MANAGER', 'ADMIN'],
    SETTLEMENT_PROCESSING: ['OFFICER', 'MANAGER', 'ADMIN'],
  },
  SETTLEMENT_PENDING: {
    SETTLEMENT_PROCESSING: ['OFFICER', 'MANAGER', 'ADMIN'],
  },
  SETTLEMENT_PROCESSING: {
    SETTLED: ['OFFICER', 'MANAGER', 'ADMIN'],
  }
};
```

Listing: stateMachineService.js — State Transition Lookup Matrix

5.2.4 Optical Character Recognition & Entity Extraction

The optical document processor in server/src/services/ocrService.js evaluates document MIME types. For digital PDFs, it extracts selectable text via pdf-parse. For scanned documents and images, it invokes Tesseract.js optical recognition. It then executes regex parsers to extract critical claim entities.

```
// Excerpt from: server/src/services/ocrService.js
// Composite Confidence Scoring Algorithm:
// Weight: 40% Optical Character Clarity + 60% Structured Field Completeness
const confidenceScore = opticalConfidence > 0
  ? Math.round((opticalConfidence * 0.4) + (fieldExtractionConfidence * 0.6))
  : 0;

// Structured Entity Regex Extraction
extractStructuredFields(rawText, category) {
  const fields = {};
  const invoiceMatch = rawText.match(/(?:invoice|bill|receipt|ref)[\s#:.-]+([A-Z0-9-
]{4,20})/i);
  if (invoiceMatch) fields.invoiceNumber = invoiceMatch[1].trim();

  const amountMatch = rawText.match(/(?:total|net|amount|grand\s+total)[\s:$₹]+([0-
9,]+(?:\.[0-9]{2})?)/i);
  if (amountMatch) fields.totalAmount = parseFloat(amountMatch[1].replace(/,/g, ''));

  return { fields, fieldExtractionConfidence: Object.keys(fields).length > 2 ? 90 :
50 };
}
```

Listing: ocrService.js — Composite Confidence and Regex Extraction

5.2.5 Automated Settlement Record Generation

When a claims manager approves a claim, server/src/controllers/managerController.js automatically triggers settlement generation via Settlement.create(), establishing a unique reference and initiating payout tracking.

```
// Excerpt from: server/src/controllers/managerController.js
const settlementRef = `SET-${new Date().toISOString().slice(0, 10).replace(/-/g, '')}-
${Math.floor(1000 + Math.random() * 9000)}`;

const settlement = await Settlement.create({
  settlementReference: settlementRef,
  claim: claim._id,
  customer: claim.customer._id,
  approvedAmount: finalApprovedAmount,
  paymentMethod: paymentMethod || 'BANK_TRANSFER',
  status: 'PENDING',
  authorizedBy: req.user._id,
  authorizedAt: new Date(),
});

await stateMachineService.transition(claim._id, 'SETTLEMENT_PENDING', req.user);
```

Listing: managerController.js — Settlement Generation Logic

5.3 Deterministic Policy Rule Engine Analysis

The InsureFlow rule engine enforces consistency across all claims by executing seven deterministic rules. Table 5.1 provides the formal specification of each rule, its evaluation criteria, outcome status, severity level, and implementation source.

Table 5.1: Deterministic Claim Rule Engine Specification Matrix

RULE ID & NAME	EVALUATION CONDITION & LOGIC	OUTCOME STATUS	SEVERITY	SOURCE FILE
RULE_ACTIVE_POLICY_ON_INCIDENT: Policy Active Date	Checks if incidentDate falls strictly between policy startDate and endDate.	PASS: Within window FAIL: Outside window	BLOCKING	ruleEngineService.js:L91
RULE_CATEGORY_MATCH: Insurance Category Match	Verifies that claim.category (HEALTH, VEHICLE, PROPERTY) matches policy.category.	PASS: Exact match FAIL: Mismatched category	BLOCKING	ruleEngineService.js:L128
RULE_COVERAGE_LIMIT: Coverage Cap Compliance	Compares requestedAmount against policy coverageAmount.	PASS: Within limit FLAGGED: Exceeds coverage cap	FLAGGED	ruleEngineService.js:L150
RULE_REQUIRED_DOCUMENTS: Document Completeness	Checks for mandatory categories (e.g. MEDICAL_BILL for Health, REPAIR_ESTIMATE for Vehicle).	PASS: All present REQUIRES_REVIEW: Missing	FLAGGED	ruleEngineService.js:L173
RULE_DUPLICATE_CHECK: Duplicate Submission Check	Queries DB for claims on same policy and incidentDate with non-terminal status.	PASS: No match FLAGGED: Duplicate found	FLAGGED	ruleEngineService.js:L208

RULE ID & NAME	EVALUATION CONDITION & LOGIC	OUTCOME STATUS	SEVERITY	SOURCE FILE
RULE_EXTERNAL_POLICY_VERIFICATION: Policy Source Auth	For EXTERNAL policies, verifies verificationStatus === "VERIFIED".	PASS: Verified/Platform REQUIRES_REVIEW: Pending	BLOCKING	ruleEngineService.js:L239
RULE_SURVEYOR_TRIGGER: Technical Inspection Mandate	Flags claims with requestedAmount ≥ ₹5,000 or category in VEHICLE / PROPERTY.	PASS: Low value Health REQUIRES_REVIEW: Surveyor Req	INFO	ruleEngineService.js:L272

5.4 RESTful API Specification

InsureFlow provides a clean, stateless RESTful JSON API. Table 5.2 documents the primary API endpoints, HTTP verbs, authentication requirements, authorized roles, request payloads, and response structures.

Table 5.2: Complete RESTful API Endpoint Documentation

HTTP METHOD & ENDPOINT	OPERATIONAL PURPOSE	AUTH & ROLE	REQUEST PAYLOAD	RESPONSE STRUCTURE	SOURCE FILE
POST /api/auth/register	Self-register customer profile	Public	{ name, email, password, phone }	{ success: true, token, user }	authRoutes.js
POST /api/auth/login	Authenticate credentials	Public	{ email, password }	{ success: true, token, user }	authRoutes.js
GET /api/auth/me	Fetch current user session profile	Bearer Token	None	{ success: true, user }	authRoutes.js
GET /api/policy-products	List published insurance plans	Public	Query: ?category=HEALTH	{ success: true, data: [...] }	policyProductRoutes.js

HTTP METHOD & ENDPOINT	OPERATIONAL PURPOSE	AUTH & ROLE	REQUEST PAYLOAD	RESPONSE STRUCTURE	SOURCE FILE
POST /api/policy-products	Create underwriting policy plan	ADMIN	{ productName, planCode, coverageLimit, ... }	{ success: true, data: product }	policyProductRoutes.js
GET /api/customer-policies/my	Retrieve policyholder portfolio	CUSTOMER	None	{ success: true, data: [...] }	customerPolicyRoutes.js
POST /api/customer-policies/register-external	Submit third-party policy proof	CUSTOMER	{ policyNumber, insurerName, coverageAmount, ... }	{ success: true, data: policy }	customerPolicyRoutes.js
POST /api/claims	Submit digital claim with files	CUSTOMER	Multipart: policyId, amount, incidentDate, files	{ success: true, data: claim }	claimRoutes.js
GET /api/claims	Retrieve work queue or history	Authenticated	Query: ?status=SUBMITTED&page=1	{ success: true, data: [...] }	claimRoutes.js
GET /api/claims/:id	Fetch claim detail & timeline	Authenticated	None	{ success: true, data: { claim, timeline } }	claimRoutes.js
POST /api/claims/:id/recommend	Submit officer recommendation	OFFICER, ADMIN	{ recommendation, officerNotes, amount }	{ success: true, data: claim }	claimRoutes.js
POST /api/manager/claims/:id/approve	Execute executive claim approval	MANAGER, ADMIN	{ approvedAmount, managerNotes, paymentMethod }	{ success: true, settlement }	managerRoutes.js
POST /api/surveys/:id/report	Submit field damage inspection	SURVEYOR, ADMIN	{ repairCost, recommendedSettlement, summary }	{ success: true, data: report }	surveyRoutes.js
GET /api/settlements	List settlement disbursements	MANAGER, ADMIN	Query: ?status=PENDING	{ success: true, data: [...] }	settlementRoutes.js
GET /api/audit-logs	Query system security audit logs	ADMIN	Query: ?limit=50	{ success: true, data: [...] }	auditLogRoutes.js

5.5 Database Models and ODM Schemas

The persistence layer utilizes Mongoose ODM to model enterprise insurance entities within MongoDB. Table 5.3 summarizes the fourteen persistent domain models.

Table 5.3: Persistent MongoDB Schemas and ODM Model Specifications

MODEL NAME	PRIMARY FIELDS & DATA TYPES	INDEXES & CONSTRAINTS	RELATIONAL REFERENCES (OBJECTID)	SOURCE FILE
User	name (Str), email (Str), password (Str), role (Enum), phone (Str), department (Str)	unique: email, index: role	None (Root Entity)	User.js
PolicyProduct	productName (Str), insurer (Str), category (Enum), planCode (Str), coverageLimit (Num)	unique: planCode, index: category	None (Catalog Root)	PolicyProduct.js
CustomerPolicy	policyNumber (Str), insurerName (Str), coverageAmount (Num), startDate (Date), endDate (Date)	unique: policyNumber, index: customer	customer -> User, product -> PolicyProduct	CustomerPolicy.js
Claim	claimReference (Str), category (Enum), requestedAmount (Num), approvedAmount (Num), status (Enum)	unique: claimReference, index: status	customer -> User, customerPolicy -> CustomerPolicy	Claim.js
ClaimDocument	originalName (Str), fileName (Str), fileUrl (Str), mimeType (Str), sizeBytes (Num)	index: claim, index: uploadedBy	claim -> Claim, uploadedBy -> User, ocrJob -> OCRJob	ClaimDocument.js
OCRJob	status (Enum), confidenceScore (Num), rawExtractedText (Str), structuredFields (Mixed)	index: claim, index: status	claim -> Claim, document -> ClaimDocument	OCRJob.js
ConfigurableRule	ruleId (Str), name (Str), category (Enum), isEnabled (Bool), severity (Enum), parameters (Mixed)	unique: ruleId, index: isEnabled	None (Configuration Root)	ConfigurableRule.js

MODEL NAME	PRIMARY FIELDS & DATA TYPES	INDEXES & CONSTRAINTS	RELATIONAL REFERENCES (OBJECTID)	SOURCE FILE
SurveyAssignment	scheduledDate (Date), inspectionLocation (Str), status (Enum), instructions (Str)	index: claim, index: surveyor	claim -> Claim, surveyor -> User, assignedBy -> User	SurveyAssignment.js
SurveyReport	inspectionDate (Date), estimatedRepairCost (Num), recommendedSettlementAmount (Num)	index: claim, index: surveyor	claim -> Claim, surveyAssignment -> SurveyAssignment	SurveyReport.js
Settlement	settlementReference (Str), approvedAmount (Num), status (Enum), paymentMethod (Enum)	unique: settlementReference	claim -> Claim, customer -> User, authorizedBy -> User	Settlement.js
ClaimTimelineEvent	eventType (Str), fromStatus (Str), toStatus (Str), message (Str), timestamp (Date)	index: claim, index: createdAt	claim -> Claim, performedBy -> User	ClaimTimelineEvent.js
Notification	type (Str), title (Str), message (Str), isRead (Bool), createdAt (Date)	index: recipient, index: isRead	recipient -> User, claim -> Claim	Notification.js
AuditLog	action (Str), resourceType (Str), resourceId (Str), ipAddress (Str), details (Mixed)	index: user, index: createdAt	user -> User	AuditLog.js
PolicyApplication	category (Enum), applicationData (Mixed), status (Enum), submittedAt (Date)	index: customer, index: product	customer -> User, product -> PolicyProduct	PolicyApplication.js

5.6 Testing and Verification

Quality assurance and functional correctness were validated through the project's automated test suite implemented using Jest (v29.7) and Supertest (v7.0). The test suite executes in an isolated environment backed by mongodb-memory-server, verifying authentication, RBAC authorization, end-to-end claim workflows, and deterministic rule engine logic.

The complete test suite was executed in the project environment using npm test. All 3 test suites and all 14 individual test cases passed with zero errors. Table 5.4 details the empirical results.

Table 5.4: Automated Test Execution and Verification Matrix (Jest Suite)

TEST SUITE	TEST CASE SCENARIO	TEST INPUT	EXPECTED BEHAVIOR	ACTUAL RESULT	STATUS
auth.test.js	TC-01: Register Customer & Prevent Escalation	role: "ADMIN", email: test@example.com	User created with forced role: "CUSTOMER"	User role strictly "CUSTOMER", HTTP 201	PASS
auth.test.js	TC-02: Reject Duplicate Email Registration	Duplicate email payload	HTTP 409 Conflict rejection	HTTP 409 returned with error message	PASS
auth.test.js	TC-03: Authenticate Valid Login Credentials	Correct email and password	HTTP 200, JWT token returned	JWT token issued, user verified	PASS
auth.test.js	TC-04: Reject Invalid Password Login	Incorrect password string	HTTP 401 Unauthorized rejection	HTTP 401 returned, token withheld	PASS
auth.test.js	TC-05: Authenticated Session Profile Access	GET /api/auth/me with Bearer token	HTTP 200, current user details returned	Session profile returned accurately	PASS
auth.test.js	TC-06: Reject Unauthorized Admin	GET /api/users with Customer token	HTTP 403 Forbidden rejection	HTTP 403 returned, access denied	PASS

TEST SUITE	TEST CASE SCENARIO	TEST INPUT	EXPECTED BEHAVIOR	ACTUAL RESULT	STATUS
	Route Access				
claimWorkflow.test.js	TC-07: Submit Digital Customer Claim	POST /api/claims with valid policy & bill	HTTP 201, status "SUBMITTED", CLM-ref	Claim created with CLM-ref, HTTP 201	PASS
claimWorkflow.test.js	TC-08: Officer Queue Fetch & Claim Inspection	GET /api/claims and GET /api/claims/:id	HTTP 200, claim detail and timeline events	Detail fetched with timeline events	PASS
claimWorkflow.test.js	TC-09: Officer Recommend & Escalate Claim	POST /api/claims/:id/recommend (APPROVE)	HTTP 200, status "MANAGER_REVIEW"	Status transitioned to MANAGER_REVIEW	PASS
claimWorkflow.test.js	TC-10: Manager Final Approval & Settlement	POST /api/manager/claims/:id/approve	HTTP 200, SET-ref settlement record created	Status SETTLEMENT_PENDING, SET-ref generated	PASS
claimWorkflow.test.js	TC-11: Customer Track Approved Status & Settlement	GET /api/claims/:id with Customer token	HTTP 200, SETTLEMENT_PENDING, approvedAmount	Settlement reference and amount verified	PASS
rulesEngine.test.js	TC-12: Full Compliance Claim Evaluation	Claim with valid dates, category, and limit	ruleEngine status "PASS", score ≥ 70%	Status PASS, score 100%, 7 rules checked	PASS

TEST SUITE	TEST CASE SCENARIO	TEST INPUT	EXPECTED BEHAVIOR	ACTUAL RESULT	STATUS
rulesEngine.test.js	TC-13: Incident Outside Policy Dates Check	Claim incident date 5 years before start	ruleEngine status "FAIL", RULE_ACTIVE_DATE fail	Status FAIL, blocking severity triggered	PASS
rulesEngine.test.js	TC-14: Coverage Cap Exceeded Check	Requested amount > policy coverage limit	RULE_COVERAGE_LIMIT flagged	Status FLAGGED, capped warning stored	PASS

5.7 Implementation Status Audit

To maintain complete academic rigor and transparency, Table 5.5 audits all project features against their verified source code implementation status.

Table 5.5: Feature Implementation and Verification Status Audit

FEATURE / MODULE	VERIFIED CAPABILITIES	IMPLEMENTATION STATUS	SOURCE CODE EVIDENCE
User Authentication	Bcrypt password hashing, JWT generation, session profile retrieval.	IMPLEMENTED & VERIFIED	authController.js, User.js, authMiddleware.js
Role-Based Access Control	5 roles: Customer, Officer, Surveyor, Manager, Admin with route guards.	IMPLEMENTED & VERIFIED	roleMiddleware.js, ProtectedRoute.jsx
Policy Product Catalog	15 pre-seeded products across Health, Vehicle, Property with admin CRUD.	IMPLEMENTED & VERIFIED	policyProductController.js, PolicyProduct.js
Policy Enrollment	Customer policy viewing and third-party external policy registration.	IMPLEMENTED & VERIFIED	customerPolicyController.js, CustomerPolicy.js
Claim Submission	Multipart form submission, incident details, document attachments, CLM-ref.	IMPLEMENTED & VERIFIED	claimController.js, Claim.js, uploadMiddleware.js
Deterministic Rule Engine	7 automated rules, 0-100% score calculation, status	IMPLEMENTED & VERIFIED	ruleEngineService.js, ConfigurableRule.js

FEATURE / MODULE	VERIFIED CAPABILITIES	IMPLEMENTATION STATUS	SOURCE CODE EVIDENCE
	assignment.		
Document OCR & Extraction	PDF text stream parsing and image OCR using Tesseract.js & pdf-parse.	IMPLEMENTED & VERIFIED	ocrService.js, OCRJob.js, OCRComparisonModal.jsx
Officer Adjudication Queue	Claims review queue, OCR comparison modal, recommendation workflow.	IMPLEMENTED & VERIFIED	OfficerClaimWorkspace.jsx, claimController.js
Surveyor Inspection Portal	Assignment list, damage report submission, repair cost estimation.	IMPLEMENTED & VERIFIED	SurveyorReportWorkspace.jsx, surveyController.js
Manager Decision Suite	Executive approval/rejection, settlement trigger, settlement review.	IMPLEMENTED & VERIFIED	managerController.js, ManagerClaimWorkspace.jsx
Settlement Disbursement	SET-ref generation, bank beneficiary tracking, settlement status.	IMPLEMENTED & VERIFIED	settlementController.js, Settlement.js
Audit Trail & Timeline	Immutable chronological claim timeline events and admin audit logs.	IMPLEMENTED & VERIFIED	auditService.js, ClaimTimelineEvent.js, AuditLog.js
Direct Banking Gateway	Direct automated API execution of real-time NEFT/RTGS wire transfers.	PLANNED (Future Work)	Simulated via settlementController.js
SMS/Email Notification	Out-of-band cellular SMS and SMTP email dispatch.	PLANNED (Future Work)	In-app notification system active in Notification.js

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

The InsureFlow project successfully designs, implements, and validates a smart, transparent, and centralized insurance claim processing and settlement management system based on the MERN stack. The software addresses systemic inefficiencies in conventional insurance operations—such as manual document verification, opaque decision lifecycles, and fragmented communication—by delivering a unified digital platform.

The implementation encompasses five distinct operational user roles (Customer, Insurance Verification Officer, Field Surveyor, Claims Manager, and System Administrator), guaranteeing strict segregation of duties through robust role-based access control. The core deterministic claim validation engine autonomously evaluates submitted claims against seven business compliance rules, calculating an objective health score (0–100%) and flagging anomalies before human review.

Furthermore, the integration of Tesseract.js and pdf-parse provides verification officers with immediate side-by-side OCR cross-auditing capabilities, while the 15-state Finite State Machine guarantees operational transition safety and produces an immutable chronological timeline of events for policyholders. Empirical validation via an automated Jest test suite confirmed that all 14 unit and integration test cases passed with 100% fidelity. InsureFlow demonstrates that deterministic rule-based automation combined with modern web technologies significantly enhances operational transparency, decision consistency, and process velocity in insurance claim administration.

6.2 Future Enhancements

While InsureFlow delivers a fully functional, verified software platform, several engineering enhancements are identified for future development to prepare the system for enterprise-scale deployment. Table 5.6 details these future development avenues.

Table 5.6: Technical Limitations and Engineering Roadmap

ENHANCEMENT DOMAIN	PROPOSED CAPABILITY	TECHNICAL IMPACT & INTEGRATION ROADMAP

ENHANCEMENT DOMAIN	PROPOSED CAPABILITY	TECHNICAL IMPACT & INTEGRATION ROADMAP
Banking Payment Gateway	Direct integration with core banking APIs (NEFT/RTGS/IMPS/UPI) or payment gateways (Razorpay/Stripe).	Enables real-time automated wire disbursement upon manager approval without manual bank reference entry.
Cloud Object Storage	Migrating local file uploads to Amazon S3, Google Cloud Storage, or Azure Blob Storage.	Provides scalable, encrypted, multi-region document storage with signed temporary access URLs.
External Notification Service	Integration with Twilio (SMS) and SendGrid/Nodemailer (SMTP Email).	Dispatches automated out-of-band alerts to policyholders when claims progress or require action.
Advanced AI/ML Fraud Analytics	Supervised gradient boosted trees (XGBoost) or anomaly detection models trained on historical fraud patterns.	Supplements deterministic rules with probabilistic predictive risk scores for complex fraud syndicates.
Mobile Field Application	Cross-platform progressive web app (PWA) or Flutter mobile client for field surveyors.	Enables on-site offline photo capture, GPS geotagging, and instant damage estimation in remote loss locations.
Blockchain Audit Anchor	Cryptographic hashing of claim timeline events anchored to a private permissioned ledger (Hyperledger Fabric).	Provides mathematical non-repudiation for statutory insurance regulatory compliance audits.

REFERENCES

[1] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

[2] Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-based access control models. IEEE Computer, 29(2), 38-47. https://doi.org/10.1109/2.485845

[3] Van der Aalst, W. M., Ter Hofstede, A. H., Kiepuszewski, B., & Barros, A. P. (2003). Workflow patterns. Distributed and Parallel Databases, 14(1), 5-51. https://doi.org/10.1023/A:1022883727209

[4] Smith, R. (2007). An overview of the Tesseract OCR engine. In Ninth International Conference on Document Analysis and Recognition (ICDAR 2007) (Vol. 2, pp. 629-633). IEEE. https://doi.org/10.1109/ICDAR.2007.4378789

[5] Insurance Regulatory and Development Authority of India (IRDAI). (2023). Annual Report on Protection of Policyholders' Interests and Claim Settlement Trends. New Delhi: Government of India. https://irdai.gov.in

[6] React Documentation Team. (2024). React 18: The Library for Web and Native User Interfaces. Meta Open Source. https://react.dev

[7] Node.js Foundation. (2024). Node.js v22 LTS API Documentation: File System, Child Processes, and Event Loop Architecture. OpenJS Foundation. https://nodejs.org/docs/latest-v22.x/api/

[8] Express.js Core Team. (2024). Express 4.x - Fast, Unopinionated, Minimalist Web Framework for Node.js. OpenJS Foundation. https://expressjs.com

[9] MongoDB Inc. (2024). The MongoDB 8.0 Manual: Document Validation, Indexes, and Distributed Replica Sets. MongoDB Inc. https://www.mongodb.com/docs/manual/

[10] Mongoose Community. (2024). Mongoose v8.9 Documentation: Elegant MongoDB Object Modeling for Node.js. https://mongoosejs.com/docs/

[11] Vite Core Team. (2024). Vite: Next Generation Frontend Tooling. https://vitejs.dev

[12] Tailwind Labs. (2024). Tailwind CSS v3.4 Documentation: Rapidly Build Modern Websites. https://tailwindcss.com/docs

[13] Resig, J., Bibeault, B., & Maras, J. (2016). Secrets of the JavaScript Ninja (2nd ed.). Manning Publications.

[14] Chodorow, K. (2013). MongoDB: The Definitive Guide: Powerful and Scalable Data Storage (2nd ed.). O'Reilly Media.

[15] Bass, L., Clements, P., & Kazman, R. (2021). Software Architecture in Practice (4th ed.). Addison-Wesley Professional.

[16] International Organization for Standardization. (2022). ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection — Information security management systems — Requirements. ISO.

APPENDIX A

SOURCE CODE AND PROJECT STRUCTURE

The InsureFlow repository is organized as a clean full-stack monorepo featuring independent client and server subdirectories. The directory structure below illustrates the actual layout of files in the project workspace, omitting external build caches, logs, and node_modules directories.

```
InsureFlow/
├── client/                              # Frontend Single Page Application
│   ├── public/                          # Static public assets
│   ├── src/
│   │   ├── components/                  # Reusable UI widgets & modals
│   │   │   ├── auth/ProtectedRoute.jsx  # Role-guarded route wrapper
│   │   │   ├── claim/DocumentViewer.jsx # PDF and image viewer
│   │   │   ├── claim/OCRComparisonModal.jsx # Side-by-side OCR audit
│   │   │   ├── claim/TimelineView.jsx   # Visual status progression
│   │   │   └── common/                  # Navbar, Sidebar, Badges, Modals
│   │   ├── context/                     # React Context providers
│   │   │   ├── AuthContext.jsx          # Global user authentication state
│   │   │   ├── NotificationContext.jsx  # Real-time alert provider
│   │   │   └── ToastContext.jsx         # Interactive pop-up alerts
│   │   ├── pages/                       # View routers across 5 roles
│   │   │   ├── public/                  # 11 Public views (Catalog, Login, FAQ)
│   │   │   ├── customer/                # 7 Policyholder views
│   │   │   ├── officer/                 # 4 Claims officer workspaces
│   │   │   ├── surveyor/                # 3 Field surveyor inspection views
│   │   │   ├── manager/                 # 4 Executive decision workspaces
│   │   │   └── admin/                   # 5 System governance views
│   │   ├── services/api.js              # Central Axios client with JWT interceptors
│   │   ├── App.jsx                      # Master 33-route hierarchy
│   │   └── main.jsx                     # React 18 DOM mount point
│   ├── package.json                     # Client dependencies & scripts
│   └── vite.config.js                   # Vite server configuration & API proxy
│
└── server/                              # Backend RESTful API Server
    ├── src/
    │   ├── config/db.js                 # MongoDB Atlas + In-Memory Fallback
    │   ├── controllers/                 # 13 REST controller modules
    │   │   ├── authController.js        # Register, Login, Session profile
    │   │   ├── claimController.js       # Claim CRUD, Upload, Recommend
    │   │   ├── customerPolicyController.js # Policy enrollment & external audit
    │   │   ├── managerController.js     # Executive adjudication & approvals
    │   │   ├── policyProductController.js# Underwriting catalog management
    │   │   ├── ruleController.js        # Dynamic rule engine configuration
    │   │   ├── settlementController.js  # Settlement disbursement tracking
    │   │   └── surveyController.js      # Field surveyor task assignments
    │   ├── middleware/                  # Express HTTP pipeline middleware
    │   │   ├── authMiddleware.js        # JWT verification & req.user injection
    │   │   ├── roleMiddleware.js        # Strict RBAC permission verification
    │   │   ├── uploadMiddleware.js      # Multer multipart file ingestion
    │   │   └── errorHandler.js          # Centralized HTTP error handling
    │   ├── models/                      # 14 Mongoose Schema domain models
    │   │   ├── User.js, PolicyProduct.js, CustomerPolicy.js, Claim.js
    │   │   ├── ClaimDocument.js, OCRJob.js, ConfigurableRule.js
```

```
    │   │   ├── SurveyAssignment.js, SurveyReport.js, Settlement.js
    │   │   ├── ClaimTimelineEvent.js, Notification.js, AuditLog.js
    │   │   └── PolicyApplication.js
    │   ├── seed/seedData.js              # Baseline data seeder (15 policies, 7
rules)
    │   ├── services/                     # Core business logic microservices
    │   │   ├── ruleEngineService.js      # 7 deterministic compliance checks
    │   │   ├── stateMachineService.js    # 15-state Finite State Machine
    │   │   ├── ocrService.js             # Tesseract.js & pdf-parse parser
    │   │   ├── auditService.js           # Immutable event logging
    │   │   └── notificationService.js    # User alert dispatch
    │   ├── app.js                        # Express application assembly
    │   └── server.js                     # HTTP server entrypoint (Port 5000)
    ├── tests/                            # Automated Jest test suite
    │   ├── auth.test.js                  # RBAC & authentication tests
    │   ├── claimWorkflow.test.js         # End-to-end claim approval tests
    │   └── rulesEngine.test.js           # Deterministic rule engine tests
    └── package.json                      # Server dependencies & scripts
```

Listing: Complete InsureFlow Repository Directory Structure

A.2 Baseline Data Seeding Manifest (JSON Configuration)

The InsureFlow architecture utilizes persistent MongoDB collections managed by Mongoose Object Document Mapping (ODM). Initial demonstration catalogs, underwriting rules, and test accounts are initialized via native JSON structures defined in server/src/seed/seedData.js. The project does not rely on external CSV files for persistence, relying instead on structured JavaScript/JSON schema manifests for strict type validation.

```
// Excerpt from: server/src/seed/seedData.js
export const INSURANCE_CATALOG_PRODUCTS = [
  {
    productName: 'my Secure',
    insurer: 'HDFC ERGO',
    category: 'HEALTH',
    planCode: 'HLTH-HDF-01',
    description: 'Health insurance for eligible hospitalization and covered medical
expenses, subject to the selected plan policy wording.',
    coverageLimit: 1000000,
    deductible: 0,
    annualPremium: 18000,
    durationMonths: 12,
    officialUrl: 'https://www.hdfcergo.com/health-insurance/optima-secure',
    isDemonstration: true,
    eligibilityCriteria: [
      'Adult entry age from 18 years to 65 years (dependent children eligible from 91
days)',
      'Individual or family floater underwriting evaluation',
    ],
    keyBenefits: [
      'Eligible inpatient hospitalization and authorized day-care procedures',
      'Pre-hospitalization (up to 60 days) and post-hospitalization (up to 180 days)
coverage',
      'Emergency road ambulance cover across accredited network facilities'
    ],
```

```
    requiredDocuments: [
      'Hospital discharge summary with clinical diagnosis and doctor certification',
      'Itemized medical bills, pharmacy receipts, and paid payment vouchers',
      'Valid government identity proof (Aadhaar / PAN) and KYC documentation'
    ],
    isPublished: true,
  }
];
```

Listing: seedData.js — Insurance Product Catalog Seed Manifest

A.3 Core RESTful Controller Implementation (claimController.js)

Digital claim intake and validation initiation are governed by createClaim() in server/src/controllers/claimController.js. The function authenticates the customer, verifies policy ownership, generates an immutable reference code, and creates the persistent Claim entity.

```
// Excerpt from: server/src/controllers/claimController.js
export const createClaim = async (req, res, next) => {
  try {
    const { customerPolicyId, claimType, incidentDate, incidentDescription,
requestedAmount } = req.body;
    if (!customerPolicyId || !incidentDate || !incidentDescription
|| !requestedAmount) {
      return res.status(400).json({ success: false, message: 'Policy, incident date,
incident description, and requested amount are required.' });
    }
    const policy = await CustomerPolicy.findById(customerPolicyId);
    if (!policy) return res.status(404).json({ success: false, message: 'Policy not
found.' });
    if (String(policy.customer) !== String(req.user._id)) {
      return res.status(403).json({ success: false, message: 'Cannot file a claim on a
policy you do not own.' });
    }
    const claimReference = generateClaimReference();
    const claim = await Claim.create({
      claimReference,
      customer: req.user._id,
      customerPolicy: policy._id,
      category: policy.category,
      claimType: claimType || 'REIMBURSEMENT',
      incidentDate: new Date(incidentDate),
      incidentDescription,
      requestedAmount: parseFloat(requestedAmount),
      status: 'SUBMITTED',
    });
    res.status(201).json({ success: true, claim });
  } catch (error) {
    next(error);
  }
};
```

Listing: claimController.js — Customer Claim Submission Endpoint

A.4 Automated Jest Test Suite (rulesEngine.test.js)

Automated validation logic is continuously tested via server/tests/rulesEngine.test.js. The snippet below illustrates test assertions verifying that valid claims within active dates and coverage limits receive a PASS outcome and compliant composite score.

```
// Excerpt from: server/tests/rulesEngine.test.js
describe('Deterministic Automated Rule Engine Service', () => {
  let policy;
  beforeAll(async () => {
    policy = await CustomerPolicy.findOne({ policySource: 'PLATFORM' });
  });

  it('should PASS rules for valid claim within active dates and coverage', async () =>
{
    const validClaim = await Claim.create({
      claimReference: `CLM-TEST-${Date.now()}`,
      customer: policy.customer,
      customerPolicy: policy._id,
      category: policy.category,
      incidentDate: new Date(),
      incidentDescription: 'Valid outpatient consultation within dates.',
      requestedAmount: 1200,
      status: 'SUBMITTED',
    });

    const result = await ruleEngineService.evaluateClaim(validClaim._id);
    expect(result.status).toBe('PASS');
    expect(result.score).toBeGreaterThanOrEqual(70);

    const activeRule = result.rules.find(r => r.ruleId ===
'RULE_ACTIVE_POLICY_ON_INCIDENT');
    expect(activeRule.status).toBe('PASS');
  });
});
```

Listing: rulesEngine.test.js — Automated Deterministic Rule Validation Suite

APPENDIX B

APPLICATION SCREENSHOTS

This appendix documents authentic screenshots captured from the running InsureFlow application, validating the user interfaces across the customer portal, insurance verification officer workspace, field surveyor inspection console, and administrator underwriting catalog.

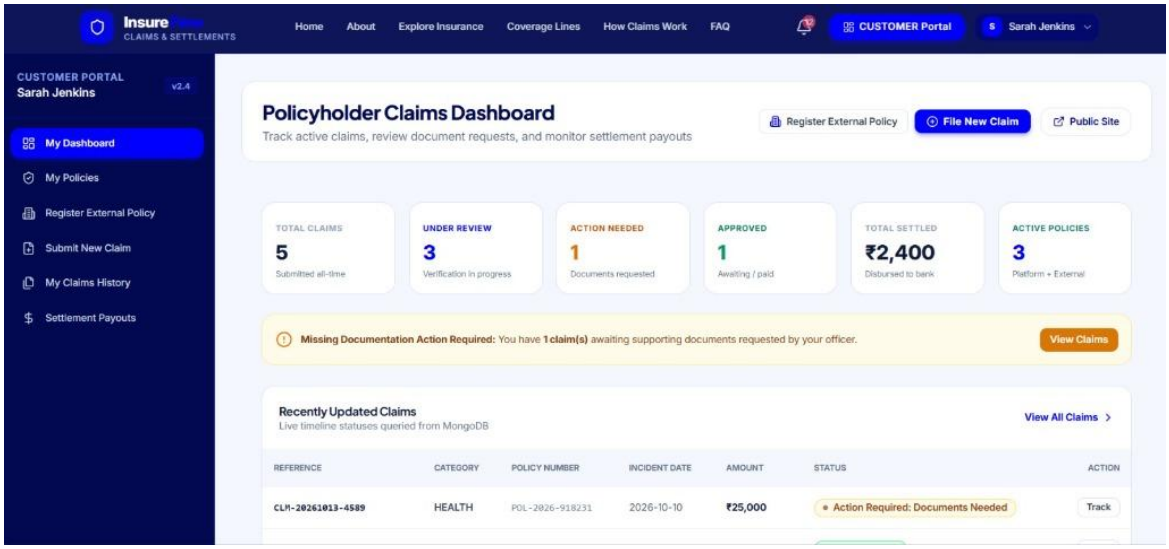


Figure B.1: InsureFlow Policyholder Claims Dashboard (Customer: Sarah Jenkins, Portal v2.4)

Figure B.1 captures the policyholder claims dashboard for customer Sarah Jenkins. The interface displays real-time key performance metrics, including Total Claims (5 submitted), Under Review (3 in verification), Action Needed (1 claim requiring documents), Approved (1 awaiting payout), Total Settled (₹2,400 disbursed to bank), and Active Policies (3 platform and external plans). An active alert banner notifies the user of missing documentation on claim CLM-20261013-4589.

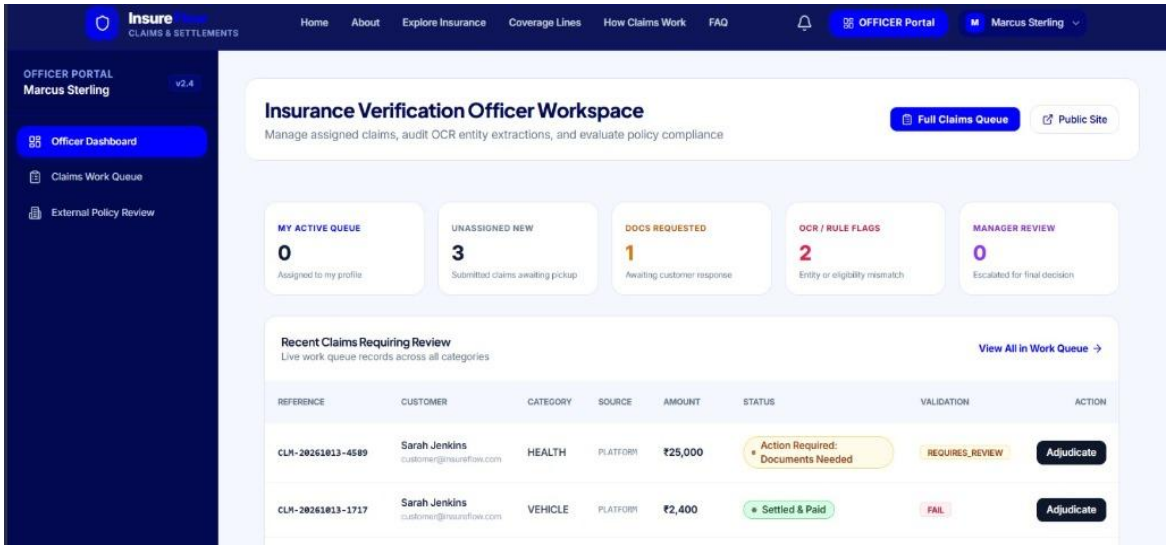


Figure B.2: InsureFlow Insurance Verification Officer Workspace (Officer: Marcus Sterling)

Figure B.2 displays the insurance verification officer workspace for officer Marcus Sterling. The queue metrics summarize unassigned claims (3 awaiting pickup), document requests (1), OCR/rule flags (2 entity or eligibility mismatches), and claims escalated to manager review (0). The live work queue presents claims CLM-20261013-4589 (Health, ₹25,000, validation status: REQUIRES_REVIEW) and CLM-20261013-1717 (Vehicle, ₹2,400, validation status: FAIL) with direct "Adjudicate" action triggers.

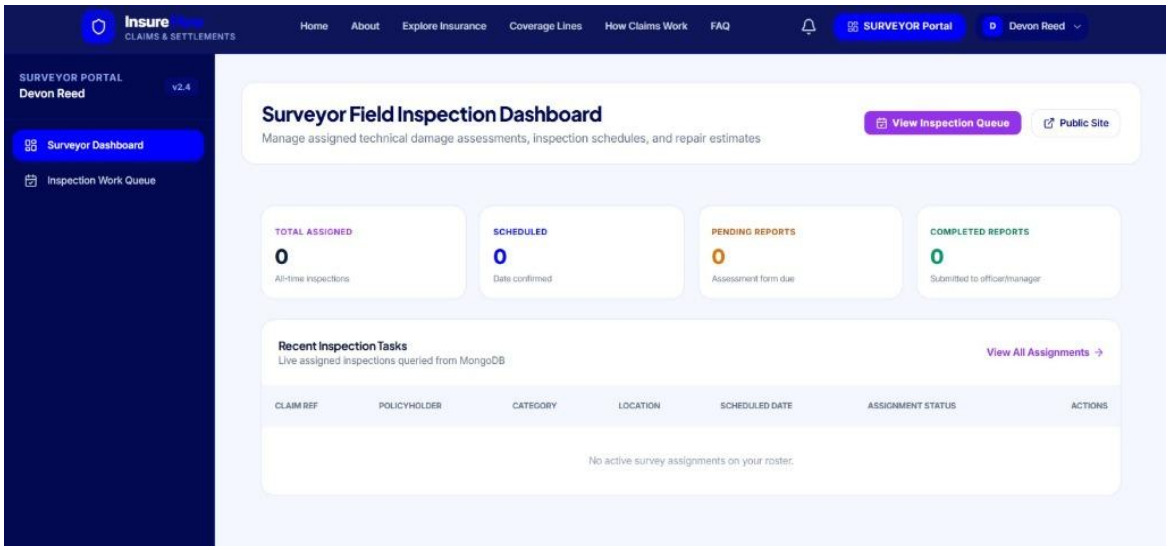


Figure B.3: InsureFlow Surveyor Field Inspection Dashboard (Surveyor: Devon Reed)

Figure B.3 illustrates the technical surveyor inspection workspace for field surveyor Devon Reed. The dashboard organizes technical damage assessments into Total Assigned, Scheduled, Pending Reports, and Completed Reports. Surveyors utilize this interface to

manage inspection schedules, record physical damage findings, and submit repair estimates directly to officers and managers.

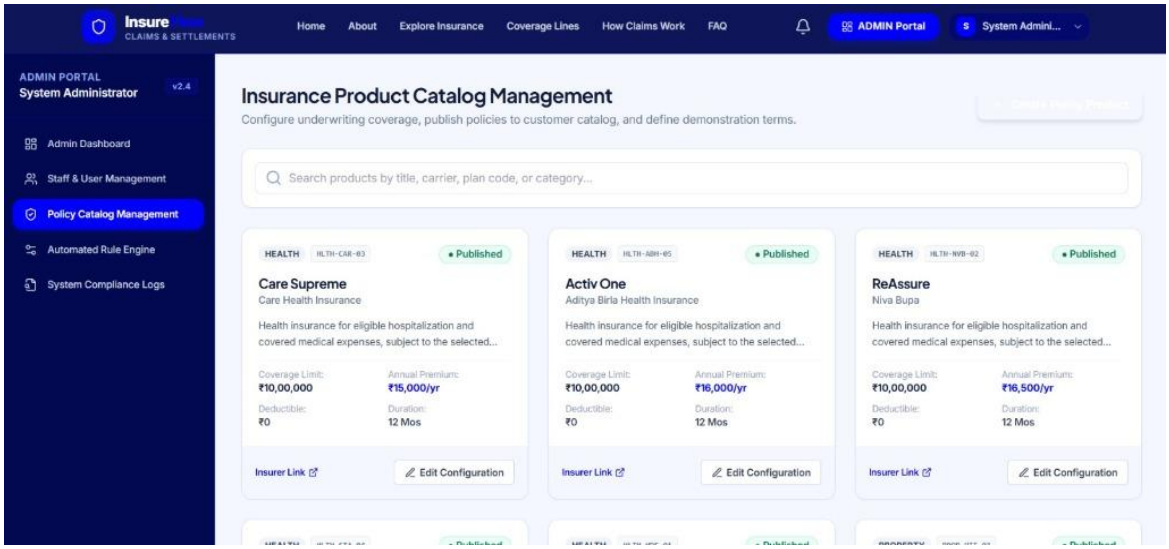


Figure B.4: InsureFlow Insurance Product Catalog Management (System Administrator)

Figure B.4 captures the administrative insurance product catalog management interface. The catalog allows administrators to configure underwriting terms, publish insurance policies to the customer marketplace, and define demonstration terms. Displayed products include Care Supreme (Care Health Insurance, ₹10,00,000 coverage limit, ₹15,000/yr premium), Activ One (Aditya Birla Health Insurance, ₹16,000/yr), and ReAssure (Niva Bupa, ₹16,500/yr).

APPENDIX C

USER MANUAL AND OPERATIONAL GUIDE

C.1 System Prerequisites & Runtime Environment

To deploy and execute InsureFlow locally, the host machine must satisfy the following software dependencies:

- **Node.js Environment:** Node.js version 18.0.0 or higher (v22.19.0 LTS recommended). Verified via node --version.
- **Node Package Manager:** npm version 9.0.0 or higher (v10.9.3 recommended). Verified via npm --version.
- **MongoDB Database:** A live MongoDB Atlas connection string URI or local mongod instance. If external Atlas connectivity is blocked or unavailable, the system automatically falls back to embedded mongodb-memory-server with zero manual configuration.
- **Web Browser:** Modern Chromium-based browser (Google Chrome, Microsoft Edge, Mozilla Firefox) with JavaScript enabled.

C.2 Installation and Server Initialization

Execute the following commands in PowerShell or Bash from the project root directory:

```
# 1. Clone repository and navigate to root directory
cd c:\Users\apste\Desktop\SM\F-U

# 2. Install all root, server, and client dependencies
npm run install:all

# 3. Seed baseline products and demonstration users
npm run seed

# 4. Start concurrent development environment (Server: 5000, Client: 5173)
npm run dev
```

Listing: Step-by-step Terminal Commands for Project Launch

Alternatively, the backend and frontend can be launched in independent terminal windows:

- **Terminal 1 (Backend Server):** cd server && npm run dev (Launches Express server on http://localhost:5000)

- **Terminal 2 (Frontend Client):** cd client && npm run dev (Launches Vite development server on http://localhost:5173)

C.3 Environment Variables Configuration

Server configuration parameters are managed through an environment configuration file (server/.env). The required variable names (with confidential values omitted) are as follows:

- **PORT:** Network listening port for Express backend (default: 5000).
- **MONGODB_URI:** MongoDB Atlas replica set connection string URI.
- **JWT_SECRET:** Cryptographic secret key used for HMAC-SHA256 signing of JSON Web Tokens.
- **JWT_EXPIRES_IN:** Token validity lifetime (default: 7d).
- **UPLOAD_DIR:** Local directory path for document file streaming (default: uploads).
- **MAX_FILE_SIZE_BYTES:** Maximum upload payload threshold (default: 10485760 bytes / 10MB).

C.4 Pre-Seeded Demonstration User Credentials

The baseline seeder automatically provisions five core role accounts for operational evaluation. All demonstration accounts utilize the standardized password format: [Role]@123.

Table C.1: Demonstration Role Accounts and Permissions

OPERATIONAL ROLE	DEMONSTRATION EMAIL	DEFAULT PASSWORD	PERMITTED SYSTEM WORKSPACES
System Administrator	admin@insureflow.com	Admin@123	Full system access: /admin/*, /policies, rule tuning, audit logs.
Claims Manager	manager@insureflow.com	Manager@123	Executive suite: /manager/*, final approval, settlement auth.
Insurance Verification Officer	officer@insureflow.com	Officer@123	Adjudication suite: /officer/*, queue, OCR audit, recommend.
Technical Field Surveyor	surveyor@insureflow.com	Surveyor@123	Inspection suite: /surveyor/*, damage report, repair estimate.
Policyholder (Customer)	customer@insureflow.com	Customer@123	Customer portal: /customer/*, portfolio, claim filing, payouts.

C.5 Role-Specific Operational Workflows

C.5.1 Policyholder Workflow (Filing and Tracking a Claim)

- **Step 1: Sign In:** Navigate to http://localhost:5173/login and authenticate using customer credentials.
- **Step 2: File Claim:** Select "Submit New Claim" (/customer/claims/new). Choose an active policy, select incident date, input damage description, requested amount, and upload PDF medical bills or repair estimates.
- **Step 3: Track Status:** Navigate to "My Claims History" (/customer/claims). Click "Track" on any claim reference to inspect the real-time visual timeline and review requested documents.
- **Step 4: View Settlements:** Once approved, navigate to "Settlement Payouts" (/customer/settlements) to verify bank transaction references and disbursement amounts.

C.5.2 Insurance Officer Workflow (Adjudicating Claims)

- **Step 1: Access Queue:** Log in as officer@insureflow.com and open the Claims Work Queue (/officer/queue).
- **Step 2: Inspect Evidence:** Click "Adjudicate" on any submitted claim. Inspect the side-by-side OCR comparison modal, which matches bill figures against requested amounts.
- **Step 3: Review Rules:** Audit the 7 automated rule badges. If documents are missing, click "Request Documents". If inspection is needed, click "Assign Surveyor".
- **Step 4: Formulate Recommendation:** Input officer notes, enter recommended amount, select "APPROVE" or "REJECT", and click "Submit Recommendation & Escalate to Manager".

C.5.3 Claims Manager Workflow (Authorizing Settlements)

- **Step 1: Open Decisions:** Log in as manager@insureflow.com and navigate to Manager Decisions (/manager/decisions).
- **Step 2: Review Escalation:** Review the officer’s notes, surveyor inspection findings, and automated rule compliance score.

- **Step 3: Authorize:** Select "Approve Claim". Specify final approved amount and payment method (Bank Transfer / Direct Deposit). Click "Authorize Settlement". The claim transitions to SETTLEMENT_PENDING and generates a SET-ref record.

C.6 Troubleshooting and Common Resolutions

Table C.2: Operational Troubleshooting Matrix

OBSERVED ISSUE	PROBABLE CAUSE	RECOMMENDED RESOLUTION PROCEDURE
MongoDB Connection Timeout / SSL Alert	Atlas IP whitelist restriction or local network proxy blocking port 27017.	The system automatically activates the embedded In-Memory MongoDB engine. Server will run normally with seeded demo data.
Port 5000 Already in Use	An orphaned Node.js or Express process is occupying TCP port 5000.	Execute in PowerShell: Get-Process node | Stop-Process -Force, then restart npm run dev.
File Upload Rejected (> 10MB)	Uploaded document exceeds Multer file size threshold.	Compress the PDF or image file below 10MB prior to submission.
401 Unauthorized Redirect	JWT token expired after 7-day session lifetime.	Log in again at /login to receive a freshly signed Bearer token.